Tugas Besar 1
IF3070 Dasar Inteligensi Artifisial
CARGO: (Constraint-Aware Revenue and Geometric Optimization for Vehicle Loading on Cargo Ships)

Disusun oleh:

Levina Nathania Bunardi - 18224057

Tiara Kusuma Wardhani - 18224059

Riantama Putra - 18224061

Program Studi Sistem dan Teknologi Informasi

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Daftar Isi

Daftar Isi **2**
Deskripsi Persoalan **4**
Pembahasan 7
2.1 Gambaran Umum Implementasi 7
2.2 Representasi dan Model Persoalan 8
2.2.1 Model Data dan Validasi Input 8
2.2.2 Representasi State 8
2.2.3 Objective Function dan Penanganan Constraint 10
2.2.4 Delta Evaluation 11
2.2.5 Neighbor 12
2.2.6 Upper Bound dan Optimum Eksak 13
2.3 Implementasi Algoritma 14
2.3.1 Steepest Ascent Hill-Climbing 14
2.3.2 Stochastic Hill-Climbing (Bonus 2) 15
2.3.3 Hill-Climbing with Sideways Move (Bonus 2) 16
2.3.4 Random Restart Hill-Climbing (Bonus 2) 16
2.3.5 Simulated Annealing 17
2.3.6 Genetic Algorithm 19
2.3.7 Bonus 1: Objective Alternatif Berbasis Urgensi ETA 20
2.3.8 Bonus 3: Multikapal 21
2.3.9 Ringkasan Heuristik yang Digunakan 21
2.4 Program dan Visualisasi 22
2.5 Hasil Eksperimen dan Analisis 23
2.5.1 Steepest Ascent Hill-Climbing 24
2.5.2 Stochastic Hill-Climbing (Bonus 2) 29
2.5.3 Hill-Climbing with Sideways Move (Bonus 2) 32
2.5.4 Random Restart Hill-Climbing (Bonus 2) 36
2.5.5 Simulated Annealing 39
2.5.6 Genetic Algorithm 44
2.5.7 Bonus 1: Objective Urgensi ETA 56
2.5.8 Bonus 3: Multikapal 61
2.5.9 Perbandingan Antaralgoritma 65

Kesimpulan dan Saran..... 68
3.1. Kesimpulan..... 68
3.2. Saran..... 69
Referensi..... 70
Lampiran..... 71
A. Pembagian Tugas..... 71
B. Github Repository..... 71
C. Gambar Run GA Lainnya..... 71

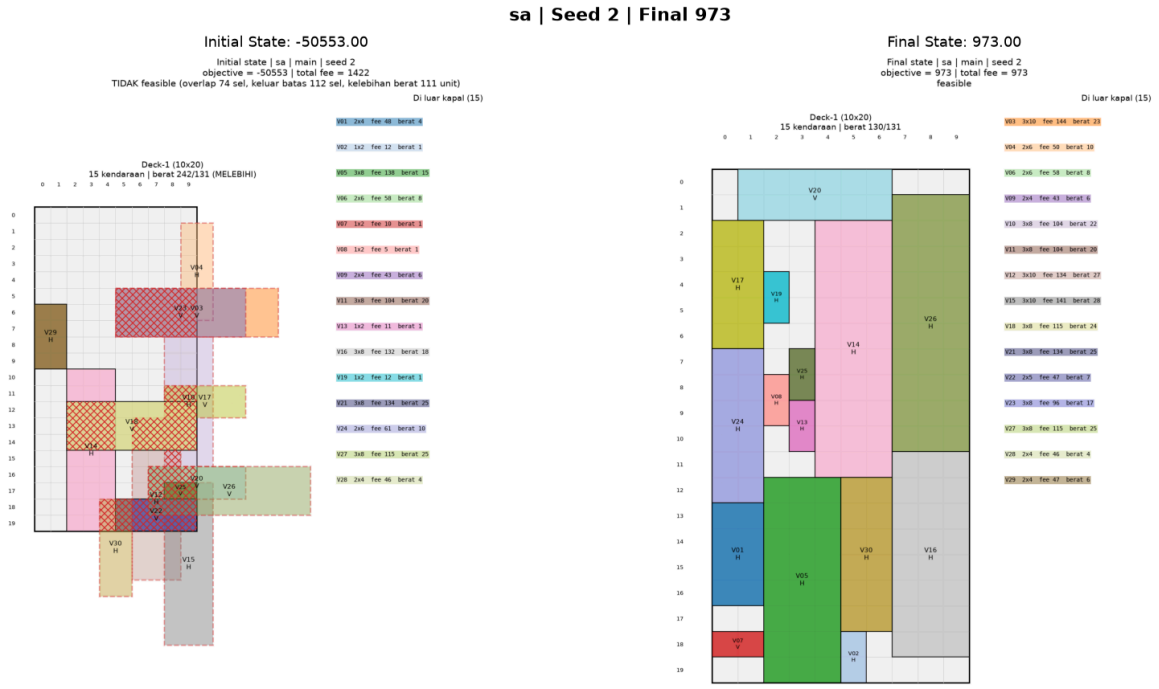
Deskripsi Persoalan

PT Intim Line mengoperasikan kapal kargo pengangkut kendaraan. Pada setiap jadwal keberangkatan, perusahaan menerima daftar kendaraan dengan dimensi, berat, tarif pengiriman (*shipping fee*), dan ETA yang berbeda-beda. Penataan yang selama ini dilakukan secara manual membuat ruang geladak tidak termanfaatkan dengan baik dan keuntungan belum tentu optimal. Persoalan ini meminta sistem yang memilih kendaraan yang diangkut serta menentukan posisi dan orientasi setiap kendaraan pada geladak sehingga total *shipping fee* maksimum



Gambar 1.1 *Gambar Kapal* (*Sumber: Wikipedia*)

Geladak dan kendaraan direpresentasikan sebagai *grid* dua dimensi. Setiap kendaraan memiliki atribut `id`, dimensi (*width x length*), orientasi (horizontal atau vertikal), *shipping fee* (juta rupiah), berat (unit), dan ETA (sisa hari). Setiap kapal memiliki dimensi geladak dan kapasitas angkut maksimum. Kendaraan boleh tidak diangkut, yaitu seluruh bagiannya berada di luar kapal.



Gambar 1.2 *Ilustrasi Grid*

Objective function: memaksimalkan total shipping fee kendaraan yang dimuat.

Constraint:

1. Tidak ada sel geladak yang ditempati lebih dari satu kendaraan (*no overlapping*).
2. Seluruh sel kendaraan berada di dalam batas geladak.
3. Total berat muatan tidak melebihi kapasitas kapal.

Ketentuan *local search*: *initial state* acak, dengan tiga langkah *neighbor* yang diizinkan, yaitu menukar dua kendaraan (di dalam maupun di luar kapal), memindahkan kendaraan ke koordinat lain, dan mengubah orientasi kendaraan. *Final state* harus memperlihatkan keadaan semua kendaraan di dalam dan di luar kapal, keadaan geladak, serta *value*-nya.

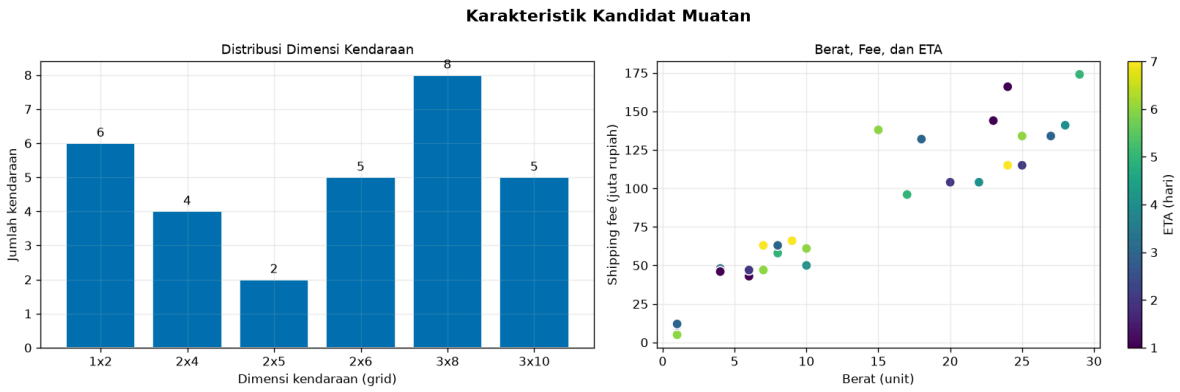
Persoalan ini merupakan gabungan *0/1 knapsack* (memilih kendaraan dengan batas berat) dan *2D packing* (menata persegi panjang tanpa tumpang tindih). Kedua persoalan tersebut NP-hard, sehingga pencarian eksak tidak praktis untuk ukuran sedang dan pendekatan *local search* menjadi pilihan yang wajar.

Instans yang digunakan. Tiga instans dibangkitkan dengan generator ber-*seed* (`src/core/generator.py`) dan disimpan pada folder `dataset/`.

Tabel 1.1 *Instans persoalan*

Instans	Kapal (lebar x panjang, kapasitas)	Kendaraan	Kegunaan
`main.json`	1 kapal: 10 x 20, kapasitas 131	30	Seluruh eksperimen wajib, varian HC, Bonus 1
`multi.json`	3 kapal: 10 x 12 (80), 8 x 10 (54), 6 x 8 (32)	40	Bonus 3 multikapal
`small.json`	1 kapal: 6 x 8, kapasitas 34	12	Pembanding optimum eksak (*brute force*)

Pada `main.json`, total berat seluruh kendaraan 382 (2,9 kali kapasitas 131) dan total luasnya 466 sel (2,3 kali luas geladak 200 sel). Dengan demikian, berat maupun luas sama-sama membatasi pemilihan kendaraan. *Shipping fee* berkisar 5-174 dengan total 2.351, berat 1-29, dan ETA 1-7 hari. Kendaraan dibangkitkan dari enam tipe berukuran tetap, yaitu motor 1 x 2, sedan 2 x 4, SUV 2 x 5, pickup 2 x 6, truk 3 x 8, dan bus 3 x 10.



Gambar 1.3 *Karakteristik Kendaraan*

Pembahasan

2.1 Gambaran Umum Implementasi

Seluruh algoritma *local search* ditulis dalam Python dan hanya memakai *standard library*. Visualisasi memakai Matplotlib. Struktur kode utama sebagai berikut.

Folder/berkas	Isi
`src/core/`	Model persoalan (`problem.py`), *objective* (`objective.py`), *state* dan *delta evaluation* (`state.py`), *upper bound* dan *brute force* (`bounds.py`), generator instans, *objective* ETA
`src/search/`	*Neighbor* (`neighbors.py`), Hill-Climbing, Simulated Annealing, Genetic Algorithm, format hasil (`result.py`)
`src/visualization/`	Gambar *state* (`state_view.py`) dan plot hasil (`plots.py`)
`src/main.py`	CLI untuk menjalankan satu algoritma
`src/experiments.py`	Runner seluruh eksperimen dan pembuat `summary.md`
`src/config.py`	Seluruh parameter *default*
`notebooks/experiments.ipynb`	Analisis dan visualisasi hasil eksperimen
`output/experiments/`	JSON dan gambar setiap *run* resmi

Semua algoritma memakai kelas `State`, fungsi *neighbor*, dan *objective* yang sama sehingga hasilnya dapat dibandingkan secara adil. Setiap algoritma mengembalikan `RunResult` yang disimpan sebagai JSON. Tabel dan gambar dibuat dari JSON tersebut, bukan dari pencarian ulang. Potongan kode pada laporan ini diringkas, yaitu *docstring*, validasi parameter, dan pencatatan untuk GUI dihilangkan. Kode lengkap tersedia pada *repository*.

2.2 Representasi dan Model Persoalan

2.2.1 Model Data dan Validasi Input

`problem.py` mendefinisikan tiga *dataclass* yang tidak dapat diubah (*frozen*), yaitu `Vehicle`, `Ship`, dan `Problem`. Fungsi `load_problem(path)` membaca JSON, lalu `parse_problem` memvalidasi isinya, yaitu id tidak kosong dan unik, dimensi bilangan bulat positif, berat dan kapasitas positif, *fee* tidak negatif, serta ETA bilangan bulat tidak negatif. Input yang tidak valid ditolak dengan `ValueError`.

```
{
  "ships":    [{"id": "Deck-1", "width": 10, "length": 20,
"max_capacity": 131}],
  "vehicles": [{"id": "V01", "width": 2, "length": 4,
"shipping_fee": 48, "weight": 4, "eta": 3}]
}
```

```
@dataclass(frozen=True, slots=True)
class Vehicle:
    """Kendaraan berukuran `width` x `length` grid (orientasi
horizontal)."""

    id: str
    width: int
    length: int
    shipping_fee: float
    weight: float
    eta: int
```

2.2.2 Representasi *State*

State adalah daftar posisi dengan satu entri untuk setiap kendaraan (indeks tetap). Notasinya sebagai berikut.

```
positions[i] = None                       # kendaraan di luar
kapal
positions[i] = (ship_index, x, y, 'H')    # anchor (x, y) =
sel kiri-atas, orientasi horizontal
positions[i] = (ship_index, x, y, 'V')    # orientasi
vertikal: lebar dan panjang ditukar
```

Orientasi `H` menempati `width` kolom dan `length` baris, sedangkan `V` menukar keduanya. Koordinat berupa bilangan bulat. Representasi ini langsung memuat keputusan "diangkut atau tidak", posisi, dan orientasi.

Kelas `State` (`src/core/state.py`) menyimpan posisi beserta agregat yang diperbarui secara bertahap (*incremental*), yaitu jumlah kendaraan pada setiap sel (*occupancy*), jumlah sel bertumpuk, jumlah sel di luar geladak, total *fee*, dan berat per kapal. Metode utamanya:

Metode	Fungsi
`set(i, placement)`	Memindahkan kendaraan ke-i dan memperbarui agregat
`apply(changes)`	Menerapkan beberapa perubahan dan mengembalikan perubahan pembatalnya
`delta(changes)`	Selisih *objective* bila perubahan diterapkan, tanpa mengubah *state*
`objective`, `is_feasible`	Nilai *objective* dan status *feasible*
`copy()`	Salinan *state*
`to_dict()`	Pemetaan semua kendaraan (di dalam/luar) dan grid setiap geladak untuk output JSON

Initial state dibangkitkan oleh `random_state`, yaitu setiap kendaraan masuk ke kapal dengan peluang `INITIAL_P_INSIDE = 0,5`, lalu kapal, *anchor*, dan orientasinya diacak. *Anchor* selalu berada di dalam geladak, tetapi badan kendaraan boleh keluar batas. Akibatnya, *initial state* hampir selalu tidak *feasible*.

```
def random_state(problem: Problem, rng: random.Random,
p_inside: float = 0.5) -> State:
    """Initial state acak: tiap kendaraan masuk ke posisi
acak dengan peluang `p_inside`, sisanya di luar."""
    return State(problem, [random_placement(problem, rng) if
rng.random() < p_inside else None
```

```
                                for _ in problem.vehicles])
```

2.2.3 *Objective Function* dan Penanganan *Constraint*

Constraint ditangani dengan penalti agar pencarian boleh melewati *state* tidak *feasible*:

$$f(s) = total_fee - P\,x\,(overlap_cells + out_of_bound_cells + weight_units)$$

dengan:

- `overlap_cells` = jumlah kelebihan penempatan pada setiap sel, yaitu Σ max(0, jumlah kendaraan di sel - 1).
- `out_of_bound_cells` = jumlah sel kendaraan yang berada di luar geladak.
- `weight_units` = Σ per kapal ⌈kelebihan berat / berat kendaraan teringan⌉.
- P = *fee* maksimum + 1 (pada `main.json`, P = 175).

Karena P lebih besar daripada *fee* kendaraan mana pun, mengeluarkan satu kendaraan yang menyebabkan pelanggaran selalu menaikkan f. Oleh karena itu, *state* yang benar-benar *local optimum* pada Hill-Climbing selalu *feasible*. Untuk *state feasible*, f sama dengan total *fee*.

```
@property
def objective(self) -> float:
    return self._fee - self._penalty * (self._overlap +
self._out_of_bounds + self._weight_units())
```

Fungsi `evaluate(problem, positions)` di `objective.py` menghitung nilai yang sama dari nol. Fungsi ini menjadi acuan kebenaran *delta evaluation* pada pengujian dan dipakai untuk laporan hasil akhir.

Incumbent feasible. Semua algoritma menyimpan arsip `FeasibleIncumbent`, yaitu *state feasible* terbaik yang pernah dikunjungi. Arsip dimulai dari kapal kosong (bernilai 0, valid karena semua kendaraan boleh tidak diangkut). `final_state` setiap algoritma adalah isi arsip ini, sehingga

hasil akhir dijamin *feasible*. Pada seluruh 39 *run* resmi, arsip selalu berisi *state* hasil pencarian, bukan kapal kosong.

```
class FeasibleIncumbent:
    def __init__(self, problem: Problem) -> None:
        self.state = State(problem)
        self.found = False

    def observe(self, state: State) -> bool:
        if state.is_feasible and (not self.found or
state.objective > self.state.objective):
            self.state = state.copy()
            self.found = True
            return True
        return False
```

2.2.4 *Delta Evaluation*

Memindahkan satu kendaraan hanya menyentuh sel lama dan sel barunya. `_update` menambah atau mengurangi *occupancy* sel-sel tersebut dan memperbarui jumlah *overlap*, sel keluar batas, *fee*, serta berat. `delta` menerapkan perubahan, membaca *objective*, lalu membatalkannya. Biaya satu evaluasi *neighbor* menjadi O (luas kendaraan), bukan O (luas geladak + jumlah kendaraan).

```
def delta(self, changes: Sequence[Change]) -> float:
    """Selisih objective jika `changes` diterapkan. State
tidak berubah setelah pemanggilan."""
    before = self.objective
    undo = self.apply(changes)
    after = self.objective
    self.apply(undo)
    return after - before
```

```
def _update(self, i: int, placement: tuple[int, int, int,
str], sign: int) -> None:
    """Tambah (sign=+1) atau keluarkan (sign=-1) kendaraan
ke-i pada `placement` dari agregat."""
    s, x, y, orientation = placement
    vehicle = self.problem.vehicles[i]
    ship = self.problem.ships[s]
    occupancy = self._occupancy[s]
```

```
    cols, rows = dimensions(vehicle, orientation)
    for cy in range(y, y + rows):
        row_inside = 0 <= cy < ship.length
        for cx in range(x, x + cols):
            if not (row_inside and 0 <= cx < ship.width):
                self._out_of_bounds += sign
                continue
            k = cy * ship.width + cx
            if sign > 0:
                if occupancy[k] > 0:
                    self._overlap += 1
                occupancy[k] += 1
            else:
                occupancy[k] -= 1
                if occupancy[k] > 0:
                    self._overlap -= 1
    self._fee += sign * vehicle.shipping_fee
    self._weight[s] += sign * vehicle.weight
```

2.2.5 *Neighbor*

Tiga langkah sesuai spesifikasi diimplementasikan di `src/search/neighbors.py`.

Langkah	Definisi
`move(i, placement)`	Memindahkan kendaraan ke koordinat/orientasi lain, ke luar kapal (`None`), atau dari luar ke dalam
`swap(state, i, j)`	Menukar *anchor* (kapal, x, y) dua kendaraan. Berlaku dalam-dalam dan dalam-luar. Setiap kendaraan mempertahankan orientasinya, sedangkan kendaraan dari luar memakai orientasi pasangannya
`rotate(state, i)`	Menukar orientasi H ↔ V pada *anchor* yang sama (hanya kendaraan nonpersegi di dalam kapal)

`all_neighbors(state)` membangkitkan seluruh *neighborhood* tanpa duplikat untuk Steepest Ascent dan Sideways Move. Pada `main.json`, ukurannya sekitar 12.000-13.000 *neighbor* (30 kendaraan x 400 penempatan,

ditambah *swap* dan *rotate*). `random_neighbor(state, rng)` memilih satu *neighbor* acak untuk Stochastic HC, SA, dan mutasi GA.

```
def random_neighbor(state: State, rng: random.Random) ->
Move:
    problem = state.problem
    inside = [i for i, p in enumerate(state.positions) if p
is not None]
    kind = rng.choice(("move", "swap", "rotate"))

    if kind == "rotate":
        rotatable = [i for i in inside if can_rotate(problem,
i)]
        if rotatable:
            return rotate(state, rng.choice(rotatable))
    elif kind == "swap" and inside and len(problem.vehicles)
> 1:
        i = rng.choice(inside)
        j = rng.choice([k for k in
range(len(problem.vehicles)) if k != i])
        return swap(state, i, j)

    i = rng.randrange(len(problem.vehicles))
    if state.placement(i) is not None and rng.random() < 0.5:
        return move(i, None)
    return move(i, random_placement(problem, rng))
```

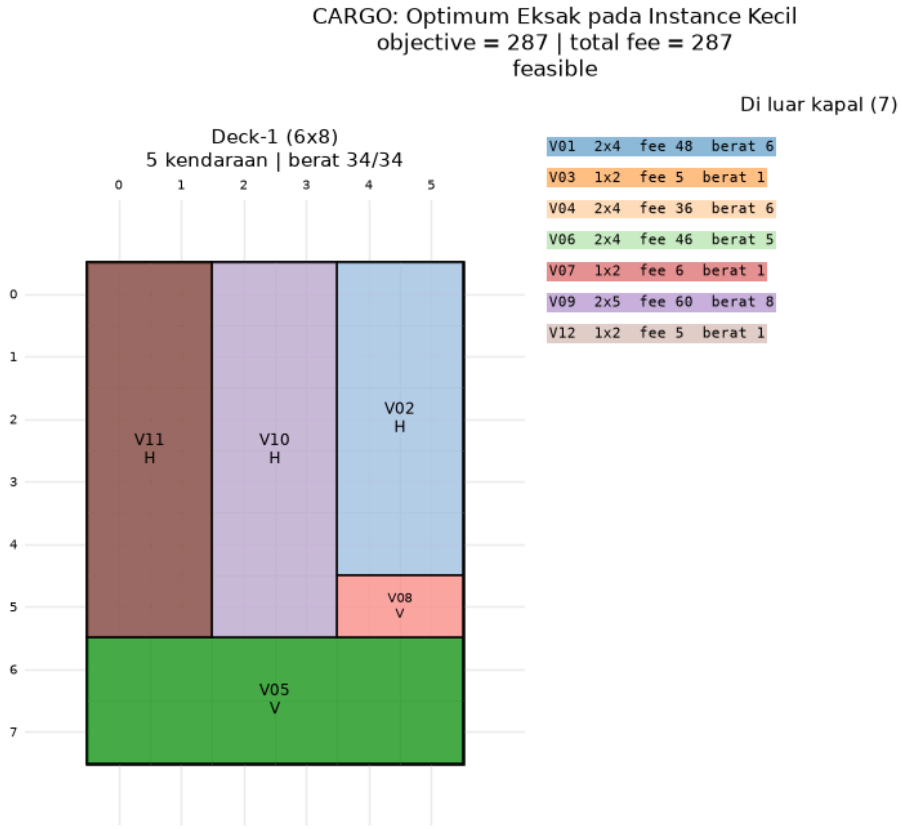
Heuristik proposal. Untuk kendaraan di dalam kapal, `random_neighbor` memberi peluang 50% untuk mengeluarkannya. Jika target dipilih seragam dari seluruh penempatan, peluang keluar hanya 1/401, sehingga kendaraan penyebab pelanggaran hampir tidak pernah dikeluarkan.

2.2.6 *Upper Bound* dan Optimum Eksak

Untuk menilai kedekatan dengan *global optimum*, `bounds.py` menyediakan dua acuan.

- `upper_bound(problem)`: relaksasi *0/1 knapsack* dua dimensi (total berat ≤ kapasitas dan total luas ≤ luas geladak) yang diselesaikan dengan *dynamic programming*. Geometri penempatan diabaikan, sehingga UB ≥ *global optimum*. $gap = \frac{UB-f}{UB}$ merupakan batas atas jarak relatif ke *global optimum*. Pada `main.json`, UB = 1.035.

- `brute_force(problem)`: *backtracking* dengan *bitmask* sel dan *branch and bound* yang memberi optimum eksak untuk instans kecil. Pada `small.json`, optimum eksak 287 dan UB 298.



Gambar 2.1 *Optimum eksak* `small.json` *(nilai 287, berat 34/34) hasil* `brute_force`

2.3 Implementasi Algoritma

Semua algoritma mengacu pada materi kuliah [1]-[5] dan berada di `src/search/`. Satu iterasi pada Steepest Ascent dan Sideways Move adalah satu perpindahan, pada Stochastic HC dan SA adalah satu *neighbor* acak yang dievaluasi, dan pada GA adalah satu generasi. Jumlah evaluasi (*delta probe* atau individu yang dinilai) dicatat terpisah karena biaya satu iterasi berbeda-beda antaralgoritma.

2.3.1 Steepest Ascent Hill-Climbing

Pada setiap iterasi, seluruh *neighbor* dievaluasi dan dipilih *neighbor* dengan Δ terbesar. Jika $\Delta \leq 0$, pencarian berhenti di *local optimum*. Jika beberapa *neighbor* sama-sama terbaik, salah satunya dipilih acak (*reservoir sampling*) agar tidak bias ke urutan enumerasi. `max_iterations = 10.000` hanya menjadi batas pengaman dan tidak pernah tercapai pada eksperimen.

```
def best_neighbor(state: State, rng: random.Random) ->
tuple[float, Move | None, int]:
    best_delta, best_move, ties, evaluated = float("-inf"),
None, 0, 0
    for move in all_neighbors(state):
        delta = state.delta(move)
        evaluated += 1
        if delta > best_delta + EPS:
            best_delta, best_move, ties = delta, move, 1
        elif delta >= best_delta - EPS:
            ties += 1
            if rng.randrange(ties) == 0:
                best_move = move
    return best_delta, best_move, evaluated
```

Steepest Ascent dan Sideways Move memakai fungsi pendakian yang sama, `_climb`. Steepest Ascent adalah kasus `max_sideways = 0`. Potongan berikut menghilangkan pencatatan untuk GUI.

```
for _ in range(max_iterations):
    delta, move, evaluated = best_neighbor(state, rng)
    climb.evaluations += evaluated
    if move is None or delta < -EPS or (delta <= EPS and
climb.sideways_moves >= max_sideways):
        climb.stop_reason = "local_optimum"
        climb.evaluation_history[-1] = climb.evaluations
        break
    state.apply(move)
    incumbent.observe(state)
    climb.feasible_history.append(incumbent.state.objective)
    climb.evaluation_history.append(climb.evaluations)
    climb.history.append(state.objective)
    if delta <= EPS:  # sideways move: nilai sama, state
berbeda
        climb.sideways_moves += 1
```

```
        if climb.sideways_moves >= max_sideways:
            climb.stop_reason = "max_sideways"
            break
```

2.3.2 Stochastic Hill-Climbing (Bonus 2)

Sesuai materi kuliah, setiap iterasi hanya membangkitkan satu *neighbor* acak dan pindah jika nilainya lebih baik ($\Delta > 0$). Pencarian berhenti setelah `max_no_improve = 20.000` *neighbor* berturut-turut tidak lebih baik, atau setelah `max_iterations = 200.000`. Kriteria ini merupakan batas *sampling*, bukan bukti *local optimum*.

```
for _ in range(max_iterations):
    move = random_neighbor(current, rng)
    if current.delta(move) > EPS:
        current.apply(move)
        no_improve = 0
    else:
        no_improve += 1
    history.append(current.objective)
    incumbent.observe(current)
    feasible_history.append(incumbent.state.objective)
    evaluation_history.append(evaluation_history[-1] + 1)
    if no_improve >= max_no_improve:
        stop_reason = "max_no_improve"
        break
```

2.3.3 Hill-Climbing with Sideways Move (Bonus 2)

Varian ini sama dengan Steepest Ascent, tetapi boleh pindah ke *neighbor* terbaik yang bernilai sama ($\Delta = 0$) agar dapat menyeberangi *shoulder*/*plateau*. Parameter `max_sideways = 100` dihitung total sepanjang run, yaitu ketika jumlah *sideways move* mencapai batas, pencarian dihentikan sesuai spesifikasi. Pencarian juga berhenti jika semua *neighbor* lebih buruk ($\Delta < 0$). Kodenya adalah `_climb` pada Subbab 2.3.1 dengan `max_sideways = 100`.

2.3.4 Random Restart Hill-Climbing (Bonus 2)

Steepest Ascent dijalankan dari *initial state* acak, lalu diulang dari *initial state* acak baru sebanyak `max_restart = 10` kali (total 11 percobaan). Hasil akhirnya adalah *state feasible* terbaik dari semua percobaan. Banyak *restart* dan banyak iterasi setiap percobaan dicatat pada `extra["restarts"]` dan `extra["iters_per_restart"]`.

```
climbs = [_climb(initial.copy(), rng, max_iterations,
max_sideways=0, observer=callback, restart=0)]
offset = len(climbs[0].history)
for restart in range(1, max_restart + 1):
    climb = _climb(random_state(problem, rng,
INITIAL_P_INSIDE), rng, max_iterations,
                   max_sideways=0, observer=callback,
step_offset=offset, restart=restart)
    climbs.append(climb)
    offset += len(climb.history)
```

```
best_climb = max(climbs, key=lambda c:
(c.incumbent.state.objective, c.incumbent.found))
```

Parameter `observer`/`callback` hanya dipakai GUI dan bernilai `None` pada eksperimen.

2.3.5 Simulated Annealing

SA mengambil satu *neighbor* acak setiap iterasi. Langkah dengan ΔE ≥ 0 selalu diterima, sedangkan langkah yang memperburuk diterima dengan peluang $e^{\Delta E/T}$. Suhu diturunkan secara geometris, T ← T x α. Pencarian berjalan tepat N = 100.000 iterasi.

```
for k in range(1, iterations + 1):
    changes = random_neighbor(current, rng)
    d = current.delta(changes)
    evaluations += 1
    improved = False
    if d >= 0:
        accept = True
    else:
        probability = math.exp(d / temperature)
```

```
            probabilities.append([k, probability])
            accept = rng.random() < probability
        if accept:
            current.apply(changes)
            if d < 0:
                accepted_worse += 1
            if current.objective > best:
                best = current.objective
                improved = True
        history.append(current.objective)
        incumbent.observe(current)
        feasible_history.append(incumbent.state.objective)
        temperature *= alpha
```

Suhu awal otomatis. Skala ΔE bergantung pada instans dan penalti, sehingga T_0 dihitung dari 100 sampel *neighbor* acak di sekitar *initial state*. Dari sampel yang memperburuk, T_0 dipilih agar penurunan rata-rata diterima dengan peluang $p_0 = 0{,}8$:

$$e^{(\text{mean}(\Delta E)/T_0)} = p_0 \Longrightarrow T_0 = \text{mean}(\Delta E) \,/\, \ln(p_0)$$

```
worse = []
for _ in range(samples):
    d = state.delta(random_neighbor(state, rng))
    if d < 0:
        worse.append(d)
if not worse:
    return 1.0
return (sum(worse) / len(worse)) / math.log(accept_prob)
```

Laju pendinginan. α dihitung agar suhu turun tepat dari T_0 ke T_{min} = 0,01 dalam N iterasi, yaitu $\alpha = (T_{min} / T_o)^{1/N}$. Dengan demikian, semua *run* memiliki jumlah iterasi dan suhu akhir yang sama walaupun T_0-nya berbeda.

Frekuensi *stuck*. *Stuck* didefinisikan secara ketat sebagai kondisi *current* tidak memiliki *neighbor* yang lebih baik. Setiap 500 iterasi (*checkpoint*), *neighborhood current* diperiksa sampai ditemukan Δ > 0 atau semua *neighbor* habis. Rangkaian *checkpoint local optimum* yang berurutan dihitung sebagai satu episode *stuck*. Pemeriksaan ini tidak memakai bilangan acak dan tidak mengubah lintasan SA, tetapi evaluasinya ikut dihitung dalam

durasi. Jumlah *window* tanpa nilai terbaik baru dicatat terpisah sebagai *stagnation*.

```
if k % stuck_window == 0:
    local_optimum = True
    for neighbor in all_neighbors(current):
        delta = current.delta(neighbor)
        evaluations += 1
        confirmation_evaluations += 1
        if delta > 0:
            local_optimum = False
            break
    local_optimum_checks.append([k, local_optimum])
    if local_optimum and not in_stuck_episode:
        stuck_iterations.append(k)
    in_stuck_episode = local_optimum
```

Tabel 2.1 *Parameter Simulated Annealing*

Parameter	Nilai
Iterasi N	100.000
T_{min}	0,01
Sampel T_0 / p_0	100 / 0,8
Interval *checkpoint stuck*	500 iterasi

2.3.6 Genetic Algorithm

Kromosom adalah daftar posisi kendaraan yang sama dengan *state* HC/SA (satu gen per kendaraan), sehingga GA memakai *objective* dan penalti yang sama. Setiap generasi: dua individu terbaik dibawa langsung (*elitism*), lalu sisa populasi diisi anak dari dua induk hasil *roulette wheel*, dengan *crossover* satu titik (p_c = 0,8) dan mutasi satu *random neighbor* pada setiap anak (p_m = 0,2).

- *Fitness* dengan *windowing*: $fit_i = f_i - \min f + \varepsilon$ ($\varepsilon = 1$), karena *objective* dapat negatif akibat penalti, sedangkan peluang *roulette* harus positif.

- *Roulette wheel*: jumlah kumulatif *fitness* dibentuk sekali setiap generasi. Setiap putaran memilih indeks pertama dengan kumulatif ≥ r ~ U(0, Σfit) memakai `bisect`.
- *Crossover* satu titik: $anak_1$ = a[0..c-1] + b[c..n-1], $anak_2$ = b[0..c-1] + a[c..n-1].
- *State* awal GA adalah individu terbaik populasi awal, sedangkan *state* akhir adalah individu *feasible* terbaik dari semua generasi.

```
def roulette_cumulative(objectives: Sequence[float], epsilon:
float = GA_EPSILON) -> list[float]:
    worst = min(objectives)
    return list(itertools.accumulate(f - worst + epsilon for
f in objectives))


def spin(cumulative: Sequence[float], rng: random.Random) ->
int:
    r = rng.uniform(0, cumulative[-1])
    return min(bisect.bisect_left(cumulative, r),
len(cumulative) - 1)


def crossover(a: State, b: State, cut: int) -> tuple[State,
State]:
    n = len(a.problem.vehicles)
    child1, child2 = a.copy(), b.copy()
    for i in range(cut, n):
        pa, pb = a.placement(i), b.placement(i)
        if pa != pb:
            child1.set(i, pb)
            child2.set(i, pa)
    return child1, child2
```

```
for generation in range(1, iterations + 1):
    ranked = sorted(range(population_size),
key=objectives.__getitem__, reverse=True)
    offspring = [population[i] for i in ranked[:elitism]]
    offspring_objectives = [objectives[i] for i in
ranked[:elitism]]
    cumulative = roulette_cumulative(objectives, epsilon)

    while len(offspring) < population_size:
        a, b = population[spin(cumulative, rng)],
population[spin(cumulative, rng)]
        if n > 1 and rng.random() < p_crossover:
```

```
            children = crossover(a, b, rng.randint(1, n - 1))
            crossovers += 1
        else:
            children = a.copy(), b.copy()
        for child in children:
            if len(offspring) == population_size:
                break
            if rng.random() < p_mutation:
                mutate(child, rng)
                mutations += 1
            offspring.append(child)
            offspring_objectives.append(child.objective)
            if incumbent.observe(child):
                best_generation = generation
            evaluations += 1
```

Tabel 2.2 *Parameter Genetic Algorithm*

Parameter	Nilai
Populasi (kontrol) / variasi	100 / 50, 100, 200
Generasi (kontrol) / variasi	1.000 / 300, 1.000, 3.000
p_c / p_m / *elitism* / ε	0,8 / 0,2 / 2 / 1

2.3.7 Bonus 1: *Objective* Alternatif Berbasis Urgensi ETA

Tanpa menambah variabel, atribut ETA yang tidak dipakai *objective* wajib dimanfaatkan sebagai bobot urgensi. Kendaraan dengan ETA kecil diasumsikan lebih mendesak:

$$w_i = fee_i \times (1 + \frac{1}{eta_i + 1})$$

Faktor pengali bernilai 2 untuk ETA 0 dan mendekati 1 untuk ETA besar (pada `main.json`, 1,125–1,5). *Objective* ETA adalah Σw_i kendaraan dimuat dikurangi penalti yang sama, dengan $P_w = max_i(w_i) + 1 = 250$. Implementasinya membuat salinan instans dengan *fee* diganti w_i, sehingga semua algoritma, penalti, dan *upper bound* dapat dipakai tanpa perubahan. *Fee* asli kendaraan yang dimuat dicatat terpisah pada `extra["shipping_fee_total"]`.

```
def urgency_weight(vehicle: Vehicle) -> float:
    """w = fee · (1 + 1/(eta + 1))."""
    return vehicle.shipping_fee * (1 + 1 / (vehicle.eta + 1))


def eta_problem(problem: Problem) -> Problem:
    vehicles = tuple(replace(v,
shipping_fee=urgency_weight(v)) for v in problem.vehicles)
    return replace(problem, vehicles=vehicles,
name=problem.name + SUFFIX)
```

2.3.8 Bonus 3: Multikapal

Input menerima daftar `ships` dengan panjang berapa pun. Elemen pertama *placement* adalah indeks kapal. *Occupancy*, sel keluar batas, dan berat dihitung per kapal, yaitu kendaraan di kapal berbeda tidak bertumpuk, dan kapasitas setiap kapal diperiksa sendiri. *Move* dapat memindahkan kendaraan ke kapal lain, sedangkan *swap* menukar *anchor* lintas kapal. *Objective* adalah total *fee* seluruh armada. Karena semua algoritma memakai API `State` yang sama, keenam algoritma langsung mendukung multikapal. Visualisasi menggambar satu panel untuk setiap geladak.

2.3.9 Ringkasan Heuristik yang Digunakan

Heuristik	Tujuan
Penalti P = *fee* maks + 1 per unit pelanggaran	Mengizinkan *state* tidak *feasible* sementara dan menjamin *local optimum* HC *feasible*
Incumbent feasible	Hasil akhir selalu *feasible* tanpa *repair* tersembunyi
Delta evaluation	Evaluasi *neighbor* O(luas kendaraan)
Peluang keluar 50% pada `random_neighbor`	Kendaraan pelanggar mudah dikeluarkan
Tie-break acak pada Steepest/Sideways	Tidak bias ke urutan enumerasi

Heuristik	Tujuan
T_0 otomatis dan α dari N	Skala suhu mengikuti instans dan suhu akhir sama untuk semua *run*
Windowing dan *elitism* pada GA	*Fitness* positif dan nilai maksimum populasi tidak turun

2.4 Program dan Visualisasi

CLI (`src/main.py`) menjalankan satu algoritma dan menyimpan JSON serta gambar ke `output/runs/`:

```
python src/main.py --algo steepest --seed 1
python src/main.py --algo sa --objective eta
python src/main.py --algo ga --population 200 --iterations
1000
python src/main.py --algo steepest --instance multi.json
python src/main.py --view
output/experiments/sa/sa/main_seed1.json
```

Opsi algoritma: `steepest` (`hc`), `stochastic`, `sideways`, `random_restart`, `sa`, dan `ga`. Parameter seperti `--max-sideways`, `--max-restart`, `--iterations`, dan `--population` dapat diubah. `--view` menggambar ulang hasil tersimpan tanpa menjalankan pencarian.

Runner eksperimen (`src/experiments.py`) menjalankan seluruh 39 *run* resmi ke `output/experiments/<grup>/<algoritma>/` dan membuat `summary.md` beserta gambar ringkasan (`--dry-run`, `--only`, `--plots-only`, `--quick`).

Visualisasi. Setiap *run* menghasilkan tiga gambar:

- `<run>_initial.png` dan `<run>_final.png` (`draw_state`): satu panel per geladak dengan kendaraan berwarna tetap, berlabel id dan orientasi. Sel bertumpuk diarsir merah, sedangkan kendaraan yang keluar geladak diberi tepi merah putus-putus. Panel kanan berisi daftar kendaraan di luar

kapal. Judul memuat *objective*, total *fee*, berat terpakai/kapasitas, dan status *feasible*.

- `<run>_plot.png` (`plot_run`): *objective* terhadap iterasi beserta garis *state feasible* terbaik dan *upper bound*. Untuk SA ditambah panel $e^{\Delta E/T}$ terhadap iterasi. Untuk GA berisi *objective* maksimum dan rata-rata populasi per generasi. Untuk Random Restart ditambah garis terbaik sejauh ini.

Notebook (`notebooks/experiments.ipynb`) membaca seluruh JSON resmi, menyusun tabel, dan membuat gambar perbandingan pada `output/analysis/`.

2.5 Hasil Eksperimen dan Analisis

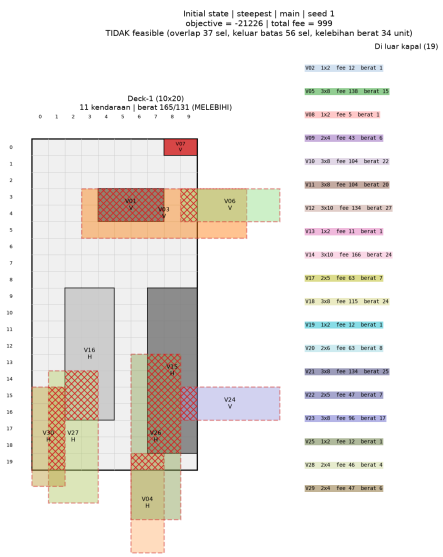
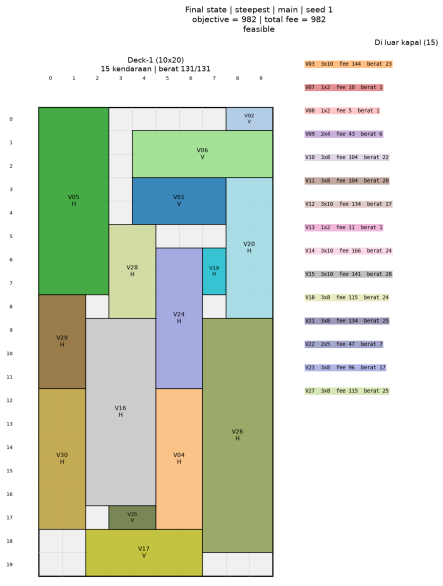
Pengaturan. Seluruh eksperimen wajib memakai `main.json` dengan *seed* 1, 2, dan 3 untuk *trial* 1-3. *Initial state* HC, varian HC, dan SA pada *seed* yang sama identik, sehingga hasilnya dapat dibandingkan langsung. Durasi adalah waktu pencarian saja (tanpa menyimpan berkas dan menggambar), diukur pada Linux 7.2.9, CPU x86_64 22 *thread*, dan Python 3.14.7. *Upper bound* `main.json` = 1.035. Semua 39 *run* menghasilkan *state* akhir *feasible*.

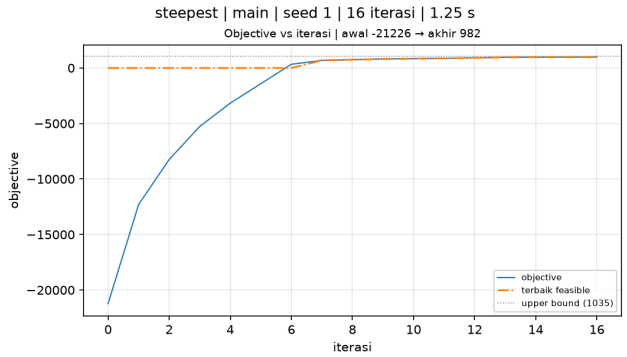
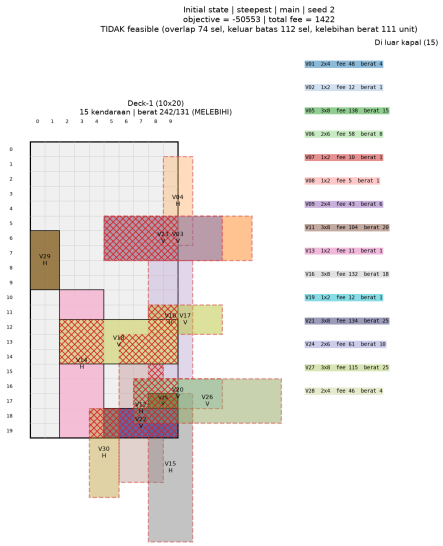
Tabel 2.3 *Daftar eksperimen*

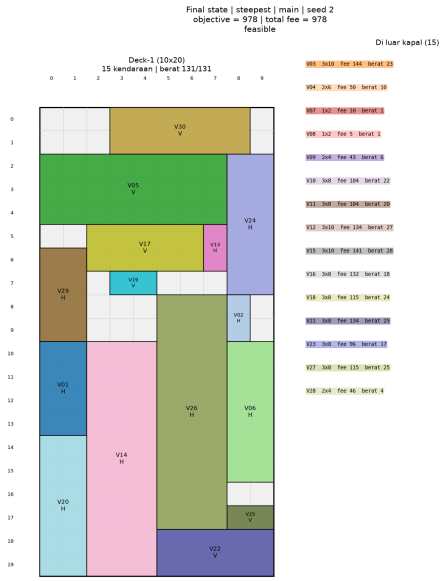
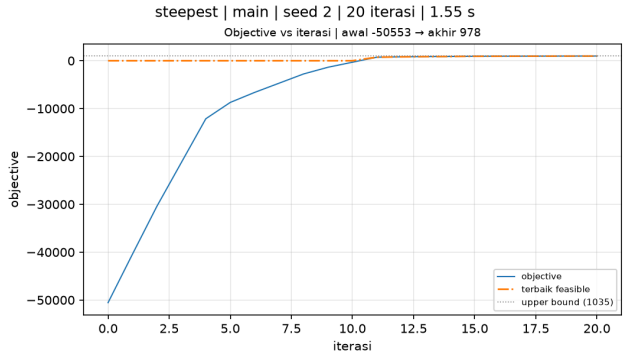
Grup	Algoritma/konfigurasi	*Run*
Wajib	*Steepest Ascent* HC	3
Wajib	*Simulated Annealing*	3
Wajib	GA populasi 100 dengan generasi 300, 1.000, 3.000	9
Wajib	GA generasi 1.000 dengan populasi 50, 100, 200	9
Bonus 2	*Stochastic, Sideways Move, Random Restart* HC	9
Bonus 1	SA dengan *objective* ETA	3
Bonus 3	*Steepest Ascent* pada armada tiga kapal	3
Total		39

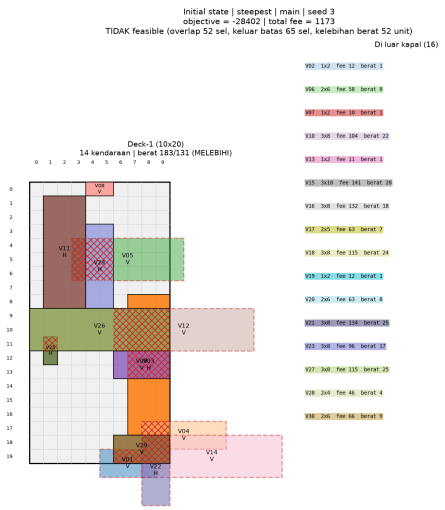
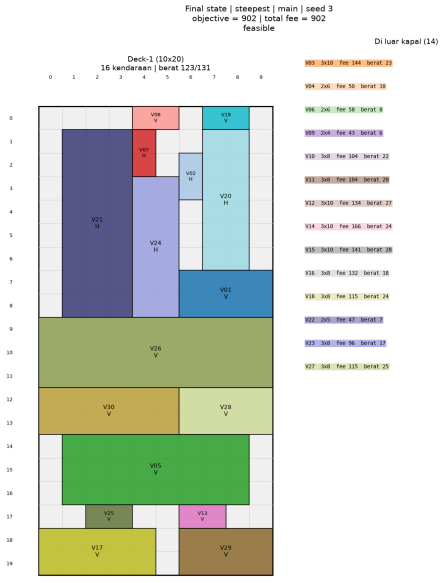
2.5.1 Steepest Ascent Hill-Climbing

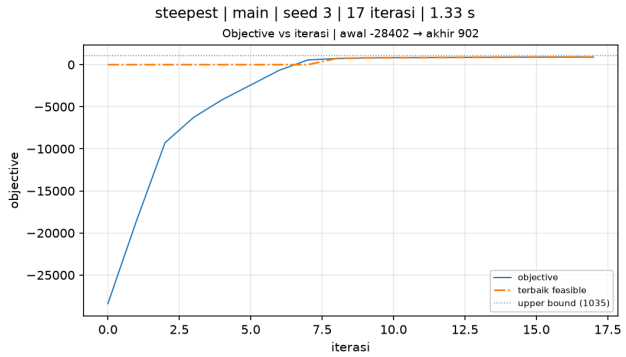
Tabel 2.4 *Visualisasi Steepest Ascent Hill-Climbing*

Trial	**Visualisasi**	**Keterangan**
1	 Gambar 2.2 *State Awal Trial 1*	*State* awal -21.226 (11 kendaraan di dalam, banyak sel bertumpuk dan keluar batas). *State feasible* pertama dicapai pada iterasi ke-7. Berhenti di *local optimum* pada iterasi ke-16 dengan nilai 982: 15 kendaraan, berat 131/131, luas terpakai 89%.
	 Gambar 2.3 *State Akhir Trial 1*	

Trial	Visualisasi	Keterangan
	 Gambar 2.4 *Plot Objective Trial 1*	
2	 Gambar 2.5 *State Awal Trial 2*	*State* awal terburuk (-50.553). *Feasible* pada iterasi ke-11, berhenti pada iterasi ke-20 dengan nilai 978: 15 kendaraan, berat 131/131, luas 88%.

Trial	Visualisasi	Keterangan
		
	Gambar 2.6 *State Akhir Trial 2*	
		
	Gambar 2.7 *Plot Objective Trial 2*	

Trial	Visualisasi	Keterangan
3	 Gambar 2.8 *State Awal Trial 3*	*State* awal -28.402. *Feasible* pada iterasi ke-8, berhenti pada iterasi ke-17 dengan nilai 902: 16 kendaraan, berat 123/131, luas 80%. Masih ada sisa kapasitas 8, tetapi tidak ada satu langkah pun yang menaikkan nilai.
	 Gambar 2.9 *State Akhir Trial 3*	

Trial	Visualisasi	Keterangan
		

Gambar 2.10 *Plot Objective Trial 3*

Tabel 2.5 *Hasil Steepest Ascent Hill-Climbing*

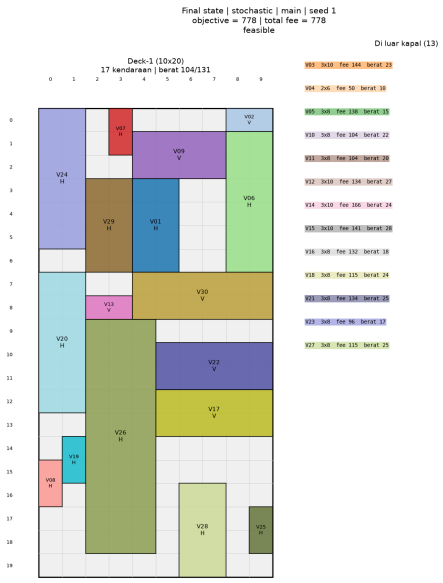
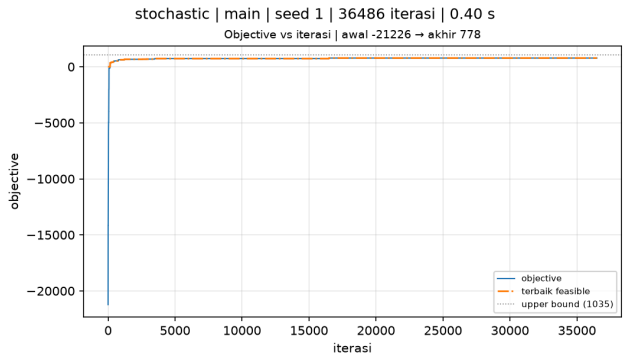
Trial	*Objective* awal	*Objective* akhir	*Gap* UB	Iterasi	Evaluasi	Durasi (s)
1	-21.226	982	5,1%	16	208.453	1,251
2	-50.553	978	5,5%	20	258.032	1,549
3	-28.402	902	12,9%	17	221.239	1,327
Rata-rata ± simpangan baku		954,0 ± 45,1	7,8%			1,376 ± 0,155

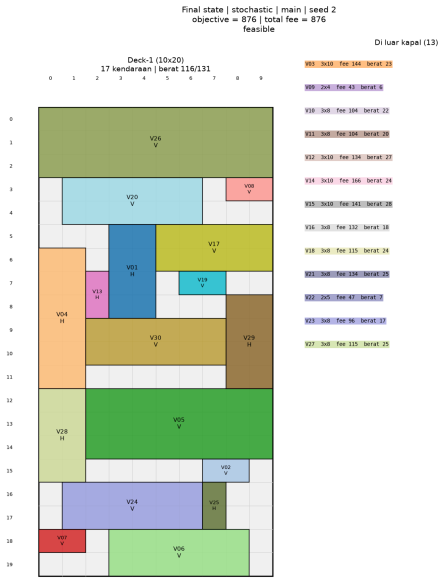
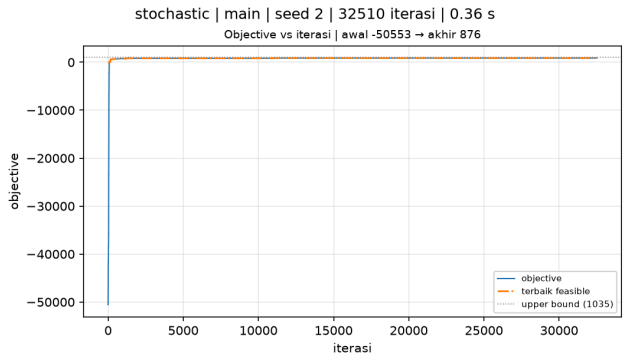
Analisis. Steepest Ascent berhenti setelah hanya 16-20 iterasi, tetapi setiap iterasi mengevaluasi sekitar 12.000-13.000 *neighbor*. Kenaikan besar pada iterasi awal berasal dari hilangnya pelanggaran karena setiap sel pelanggaran bernilai -175. Setelah *feasible*, kenaikannya jauh lebih kecil hingga mencapai *local optimum*. Hasil sangat bergantung pada *initial state*: dua *trial* mencapai 978-982 (berat penuh 131/131), sedangkan *trial* 3 terjebak di 902 dengan sisa kapasitas berat dan ruang yang tidak dapat diisi oleh satu langkah. Ini adalah ciri khas Hill-Climbing, yaitu cepat, tetapi tidak dapat keluar dari *local optimum*.

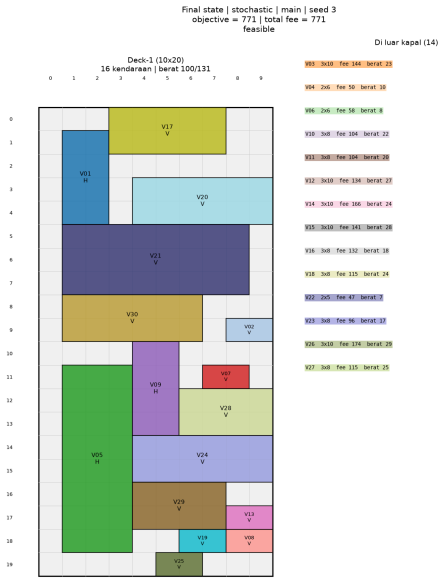
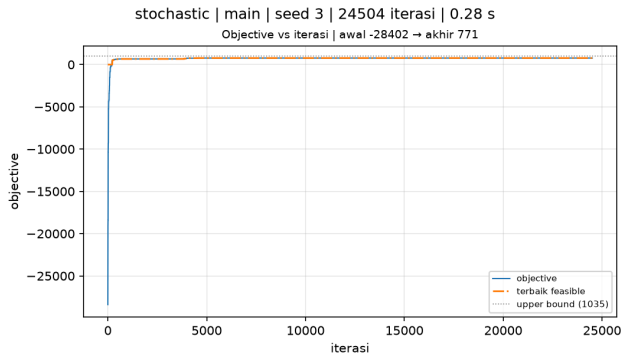
2.5.2 Stochastic Hill-Climbing (Bonus 2)

Initial state setiap *trial* sama dengan Gambar 2.2, 2.5, dan 2.8.

Tabel 2.6 *Visualisasi Stochastic Hill-Climbing*

Trial	**Visualisasi**	**Keterangan**
1	 Gambar 2.11 *State Akhir Trial 1*  Gambar 2.12 *Plot Objective Trial 1*	*Feasible* pada iterasi ke-151. Hanya 38 langkah yang diterima. Nilai akhir 778 dicapai pada iterasi ke-16.486, lalu berhenti pada iterasi ke-36.486 setelah 20.000 proposal tanpa perbaikan. Berat hanya 104/131.

Trial	Visualisasi	Keterangan
2	 Gambar 2.13 *State Akhir Trial 2*  Gambar 2.14 *Plot Objective Trial 2*	53 langkah diterima. Nilai 876 dicapai pada iterasi ke-12.510, berhenti pada iterasi ke-32.510. Berat 116/131, luas 80%.

Trial	Visualisasi	Keterangan
3	 Gambar 2.15 *State Akhir Trial 3*  Gambar 2.16 *Plot Objective Trial 3*	46 langkah diterima. Nilai 771 dicapai pada iterasi ke-4.504, berhenti pada iterasi ke-24.504. Berat 100/131, luas 69%.

Tabel 2.7 *Hasil Stochastic Hill-Climbing*

Trial	*Objective* awal	*Objective* akhir	*Gap* UB	Iterasi	Durasi (s)
1	-21.226	778	24,8%	36.486	0,405
2	-50.553	876	15,4%	32.510	0,365
3	-28.402	771	25,5%	24.504	0,277

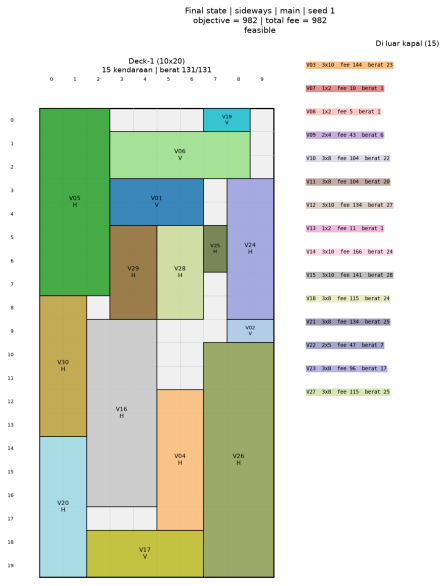
Trial	*Objective* awal	*Objective* akhir	*Gap* UB	Iterasi	Durasi (s)
Rata-rata ± simpangan baku		808,3 ± 58,7	21,9%		0,349 ± 0,065

Analisis. Stochastic HC adalah varian tercepat (0,35 detik), tetapi hasilnya paling rendah. Hanya 38-53 dari 24.504-36.486 proposal yang diterima. Ketika *state* sudah padat, *neighbor* yang membaik hanya sedikit di antara sekitar 12.000 kemungkinan. Batas 20.000 proposal tanpa perbaikan tercapai sebelum *neighbor* tersebut ditemukan, sehingga pencarian berhenti dengan sisa kapasitas 15-31 unit.

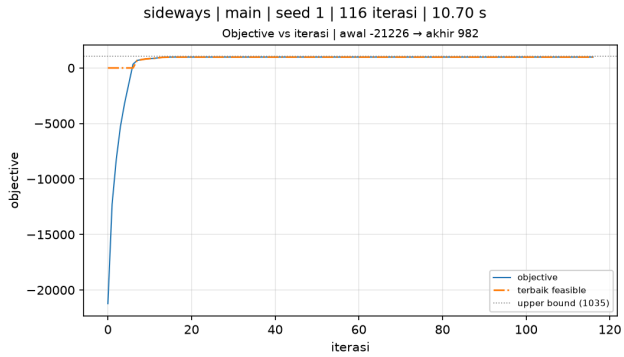
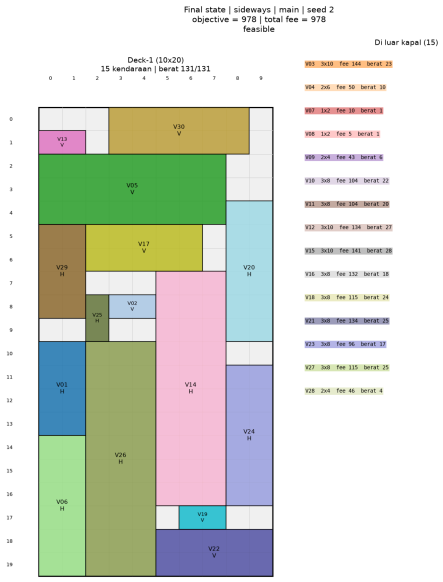
2.5.3 Hill-Climbing with Sideways Move (Bonus 2)

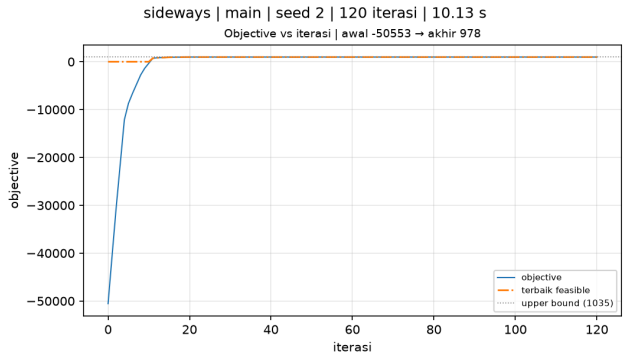
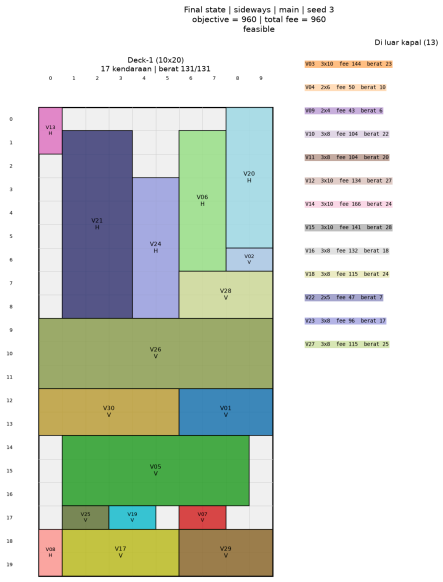
Parameter `max_sideways = 100`. *Initial state* sama dengan Gambar 2.2, 2.5, dan 2.8.

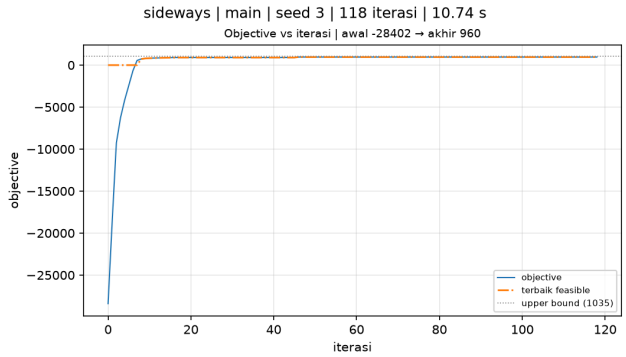
Tabel 2.8 *Visualisasi Hill-Climbing with Sideways Move*

Trial	Visualisasi	Keterangan
1		Nilai 982 dicapai pada iterasi ke-16, sama dengan Steepest Ascent. Setelah itu, 100 *sideways move* tidak menemukan perbaikan, dan pencarian berhenti karena batas *sideways* pada iterasi ke-116.

Gambar 2.17 *State Akhir Trial 1*

Trial	Visualisasi	Keterangan
	 Gambar 2.18 *Plot Objective Trial 1*	
2	 Gambar 2.19 *State Akhir Trial 2*	Sama dengan Steepest Ascent (978 pada iterasi ke-20), lalu berhenti karena batas *sideways* pada iterasi ke-120.

Trial	Visualisasi	Keterangan
	 Gambar 2.20 *Plot Objective Trial 2*	
3	 Gambar 2.21 *State Akhir Trial 3*	*Sideways move* berhasil keluar dari *plateau* tempat Steepest Ascent berhenti (902): nilai naik menjadi 960 pada iterasi ke-46 dengan 17 kendaraan dan berat 131/131. Berhenti karena batas *sideways* pada iterasi ke-118.

Trial	Visualisasi	Keterangan
		

Gambar 2.22 *Plot Objective Trial 3*

Tabel 2.9 *Hasil Hill-Climbing with Sideways Move*

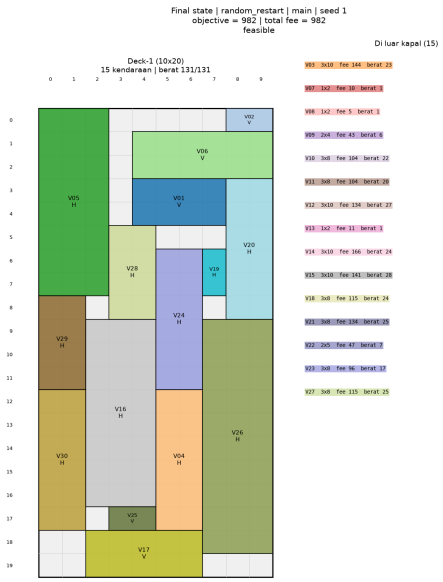
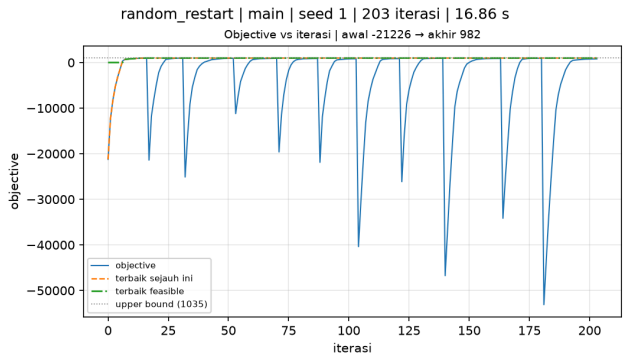
Trial	*Objective* akhir	*Gap* UB	Iterasi	Jumlah *sideways*	Alasan berhenti	Durasi (s)
1	982	5,1%	116	100	`max_sideways`	10,700
2	978	5,5%	120	100	`max_sideways`	10,127
3	960	7,2%	118	100	`max_sideways`	10,743
Rata-rata ± simpangan baku	973,3 ± 11,7	6,0%				10,523 ± 0,344

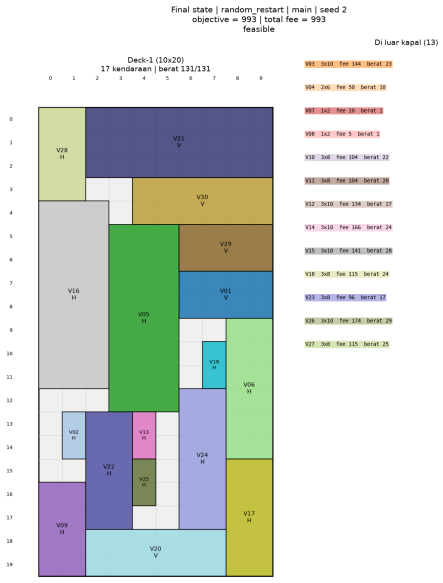
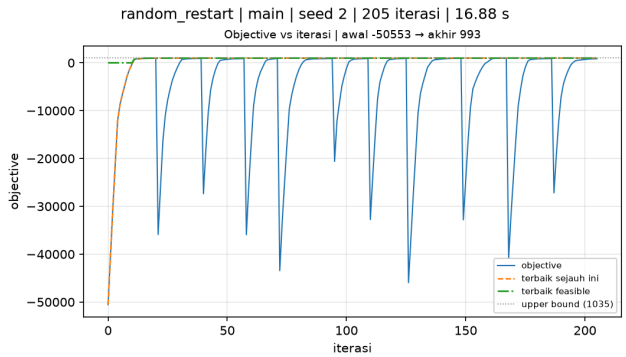
Analisis. Ruang pencarian memiliki *plateau* yang luas: memindahkan kendaraan ke sel kosong atau menukar kendaraan setara tidak mengubah total *fee*. *Sideways move* hanya bermanfaat pada *trial* 3 (+58 dibanding Steepest). Pada dua *trial* lain, 100 *sideways move* tidak menemukan jalan naik. Biayanya sekitar 7,6 kali Steepest (10,5 detik dibanding 1,4 detik) karena setiap *sideways move* tetap memindai seluruh *neighborhood*. Simpangan baku turun dari 45,1 menjadi 11,7.

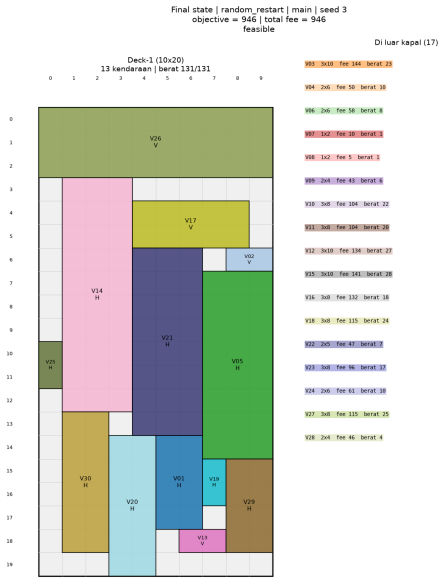
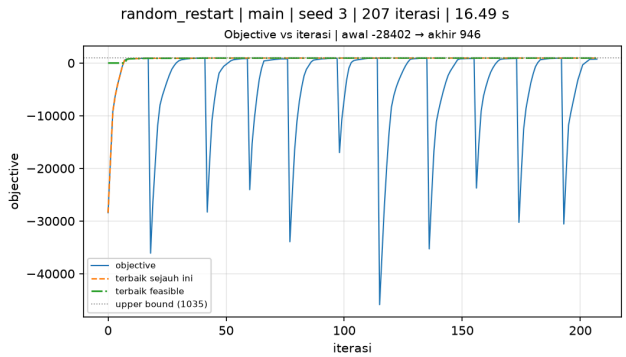
2.5.4 Random Restart Hill-Climbing (Bonus 2)

Parameter `max_restart = 10` (11 percobaan). *Initial state* percobaan pertama sama dengan Gambar 2.2, 2.5, dan 2.8.

Tabel 2.10 *Visualisasi Random Restart Hill-Climbing*

Trial	**Visualisasi**	**Keterangan**
1	 Gambar 2.23 *State Akhir Trial 1*  Gambar 2.24 *Plot Objective Trial 1*	Hasil terbaik 982 berasal dari percobaan pertama (sama dengan Steepest Ascent). Sepuluh *restart* menghasilkan 792–973, sehingga tidak ada perbaikan.

Trial	Visualisasi	Keterangan
2	 Gambar 2.25 *State Akhir Trial 2*	Nilai terbaik seluruh eksperimen, 993, diperoleh pada *restart* ke-8: 17 kendaraan, berat 131/131, luas 90%.
	 Gambar 2.26 *Plot Objective Trial 2*	

Trial	Visualisasi	Keterangan
3	 Gambar 2.27 *State Akhir Trial 3*  Gambar 2.28 *Plot Objective Trial 3*	Percobaan pertama hanya 902, sedangkan *restart* ke-1 memberi 946, nilai terbaik pada *trial* ini.

Tabel 2.11 *Hasil Random Restart Hill-Climbing*

Trial	*Objective* akhir	*Gap* UB	Banyak *restart*	Iterasi per percobaan	*Objective* per percobaan	Durasi (s)
1	982	5,1%	10	16, 14, 20, 17, 16, 15, 17, 17, 23, 16, 22	982, 973, 914, 902, 872, 818, 858, 927, 877, 941, 792	16,864

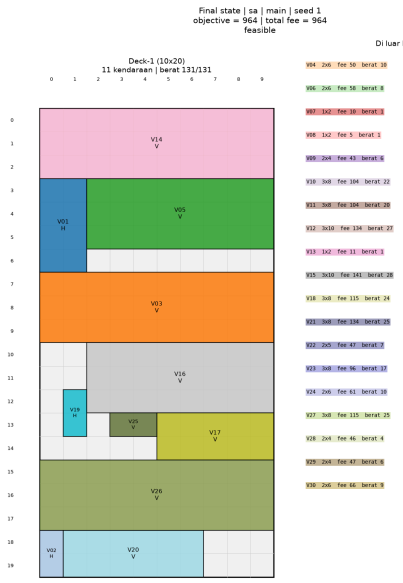
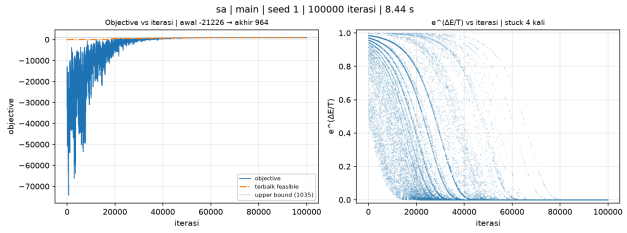
Trial	*Objective* akhir	*Gap* UB	Banyak *restart*	Iterasi per percobaan	*Objective* per percobaan	Durasi (s)
2	993	4,1%	10	20, 18, 17, 13, 22, 14, 15, 22, 18, 18, 18	978, 903, 860, 854, 877, 872, 893, 922, 993, 876, 837	16,876
3	946	8,6%	10	17, 23, 17, 16, 20, 16, 20, 19, 17, 18, 14	902, 946, 833, 785, 883, 940, 916, 860, 861, 926, 742	16,493
Rata-rata ± simpangan baku	973,7 ± 24,6	5,9%				16,745 ± 0,218

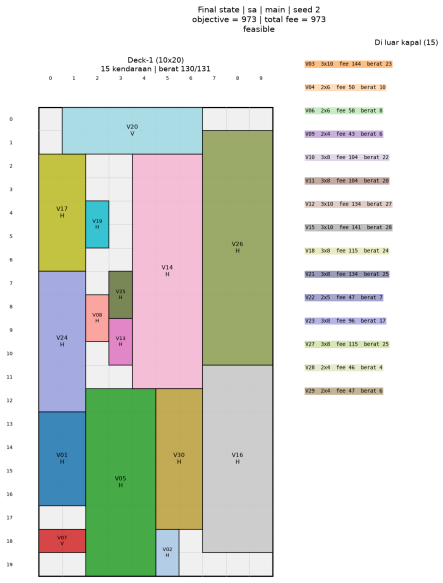
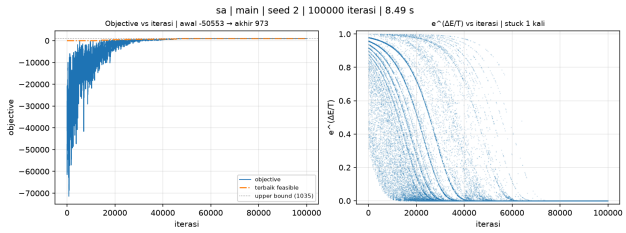
Analisis. Nilai *local optimum* tiap percobaan tersebar lebar (742-993) walaupun setiap percobaan hanya 13-23 iterasi. Hal ini menunjukkan bahwa *landscape* persoalan sangat *rugged* dan kualitas Steepest Ascent ditentukan oleh *initial state*. *Restart* menaikkan hasil pada dua dari tiga *trial* dan menghasilkan nilai terbaik keseluruhan (993). Biayanya sekitar 12 kali Steepest Ascent, sebanding dengan 11 percobaan yang dijalankan.

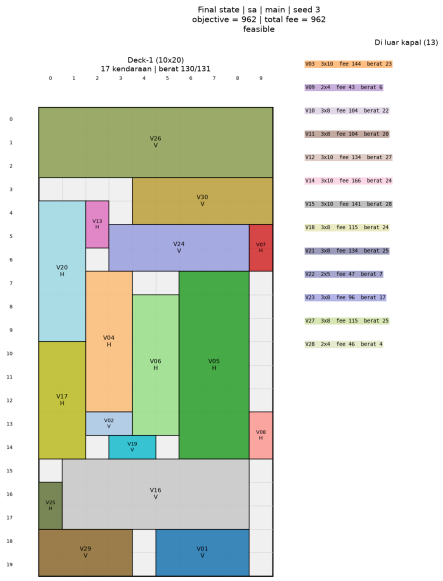
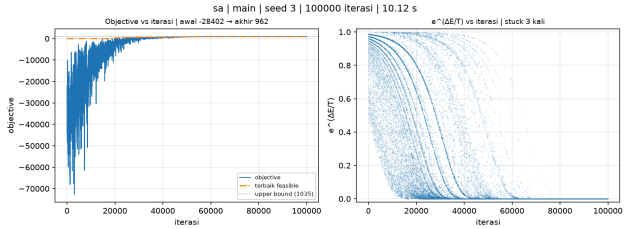
2.5.5 Simulated Annealing

Pada plot, panel kiri menampilkan *objective current* dan *state feasible* terbaik, sedangkan panel kanan menampilkan $e^{\Delta E/T}$ untuk setiap langkah yang memperburuk. *Initial state* sama dengan Gambar 2.2, 2.5, dan 2.8.

Tabel 2.12 *Visualisasi Simulated Annealing*

Trial	**Visualisasi**	**Keterangan**
1	 Gambar 2.29 *State Akhir Trial 1*  Gambar 2.30 *Plot Objective dan* $e^{\Delta E/T}$ *Trial 1*	Nilai akhir 964 dengan 11 kendaraan, berat 131/131, luas 87%. *State feasible* pertama yang lebih baik dari kapal kosong baru ditemukan pada iterasi ke-16.997. Nilai akhir dicapai pada iterasi ke-60.862. Ada 4 episode *stuck* (iterasi ke-59.000, 63.000, 66.500, 70.500).

Trial	Visualisasi	Keterangan
2	 Gambar 2.31 *State Akhir Trial 2*	Nilai akhir 973 dengan 15 kendaraan, berat 130/131. *Feasible* pada iterasi ke-14.239, nilai akhir pada iterasi ke-65.868. Satu episode *stuck* (iterasi ke-66.000) yang berlangsung hingga akhir.
	 Gambar 2.32 *Plot Objective dan* $e^{\Delta E/T}$ *Trial 2*	

Trial	Visualisasi	Keterangan
3	 Gambar 2.33 *State Akhir Trial 3*	Nilai akhir 962 dengan 17 kendaraan, berat 130/131. *Feasible* pada iterasi ke-19.771, nilai akhir pada iterasi ke-62.170. Tiga episode *stuck* (iterasi ke-62.500, 68.000, 70.000).
	 Gambar 2.34 *Plot Objective dan* $e^{\Delta E/T}$ *Trial 3*	

Tabel 2.13 *Hasil Simulated Annealing*

Trial	*Objective* awal	*Objective* akhir	*Gap* UB	Iterasi	Frekuensi *stuck*	T_0	Langkah memburuk diterima	Durasi (s)
1	-21.226	964	6,9%	100.000	4	12.153,5	8.907 dari 81.067 (11,0%)	8,435
2	-50.553	973	6,0%	100.000	1	8.142,4	8.272 dari 80.122 (10,3%)	8,488

Trial	*Objective* awal	*Objective* akhir	*Gap* UB	Iterasi	Frekuensi *stuck*	T_0	Langkah memburuk diterima	Durasi (s)
3	-28.402	962	7,1%	100.000	3	13.226,0	9.076 dari 81.111 (11,2%)	10,121
Rata-rata ± simpangan baku		966,3 ± 5,9	6,6%		2,67			9,015 ± 0,959

Analisis:

- Perilaku suhu. T_0 besar (8.142-13.226) karena sampel ΔE diambil dari *initial state* yang penuh penalti. Suhu baru turun di bawah 100 pada iterasi ke-32.000-35.000, sehingga selama sekitar sepertiga awal iterasi SA hampir menerima semua langkah dan *current* masih bernilai negatif hingga iterasi ke-33.872-37.805. Penyempurnaan utama terjadi di antara iterasi ke-40.000 dan 60.000. Setelah itu, peluang penerimaan praktis nol dan *current* tidak berubah lagi pada 27.643-34.132 iterasi terakhir, sehingga hampir sepertiga akhir iterasi tidak lagi mengubah hasil.
- Plot $e^{\Delta E/T}$. Titik membentuk pita-pita menurun karena nilai ΔE yang sama (kelipatan penalti per sel) muncul berulang-ulang. Untuk ΔE yang sama, peluang turun seiring turunnya T.
- *Stuck* dan *stagnation*. Episode *stuck* hanya 1-4 dan seluruhnya muncul setelah iterasi ke-59.000 ketika suhu sudah rendah. Sebaliknya, *stagnation* tercatat 179-185 dari 200 *window*. Tidak adanya nilai terbaik baru belum berarti *current* berada di *local optimum*.
- Kualitas dan konsistensi. Rata-rata SA (966,3) lebih tinggi daripada Steepest Ascent (954,0) dari *initial state* yang sama. SA kalah tipis pada *trial* 1-2, tetapi jauh lebih baik pada *trial* 3 (962 dibanding 902). Simpangan baku SA (5,9) adalah yang terkecil dari semua algoritma: penerimaan langkah buruk pada suhu tinggi mengurangi pengaruh *initial state*.
- Durasi. Sekitar 91% evaluasi SA (970.691-1.090.585 dari total sekitar 1,1 juta) berasal dari pemeriksaan *stuck*, bukan dari 100.101 evaluasi pencarian. Sebagai pembanding, SA yang dijalankan ulang dengan pemeriksaan

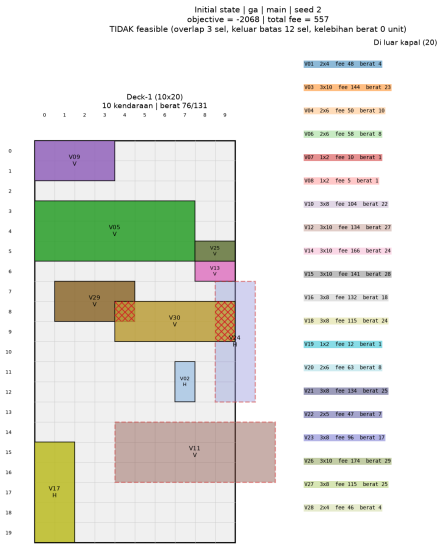
stuck dinonaktifkan (`stuck_window = 10**9`) menghasilkan *state* akhir yang identik dalam 1,20 / 1,25 / 1,27 detik. Jadi, sekitar 85-87% durasi SA pada Tabel 2.13 adalah biaya pengukuran frekuensi *stuck*.

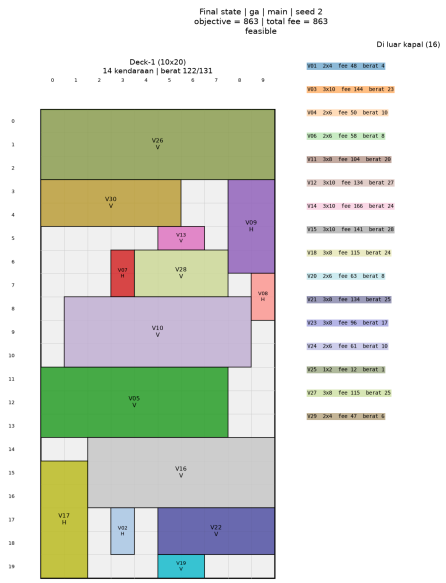
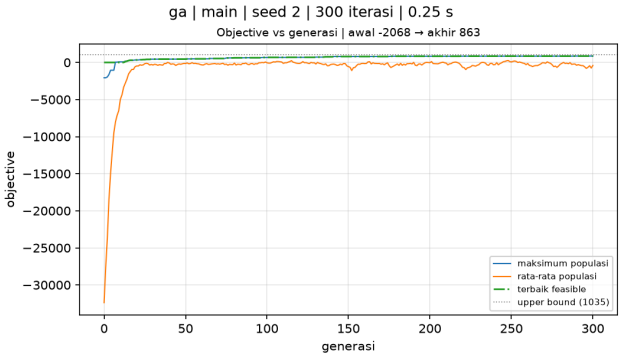
2.5.6 Genetic Algorithm

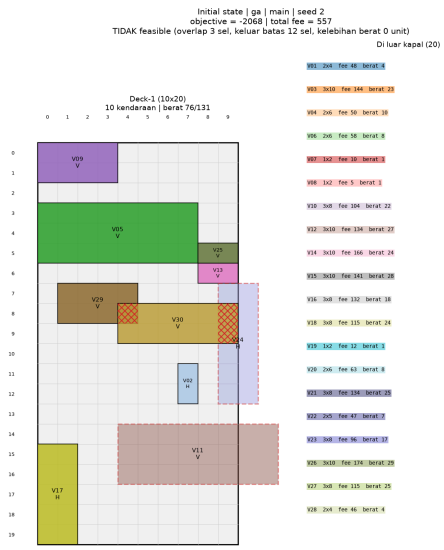
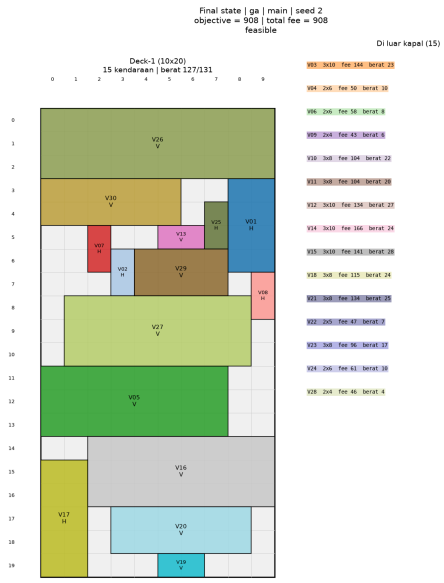
Pada plot GA, garis biru adalah *objective* maksimum populasi, garis jingga adalah rata-rata populasi, dan garis hijau putus-putus adalah individu *feasible* terbaik. Badan laporan menampilkan *seed* terbaik setiap konfigurasi, sedangkan gambar *seed* lainnya terdapat pada Lampiran B. Konfigurasi kontrol (populasi 100, generasi 1.000) dijalankan terpisah pada kedua kelompok dan menghasilkan nilai *objective* yang identik, sehingga gambarnya ditampilkan sekali.

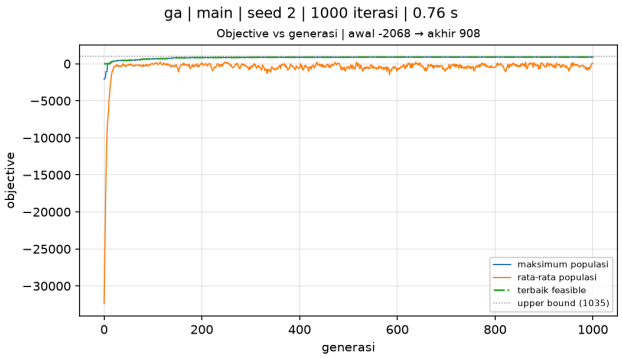
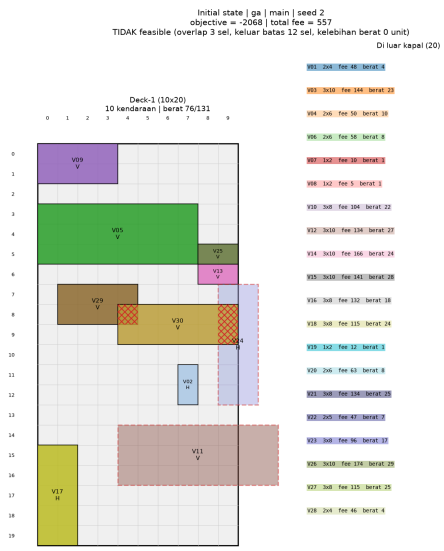
2.5.6.1 Variasi Banyak Generasi (Populasi 100)

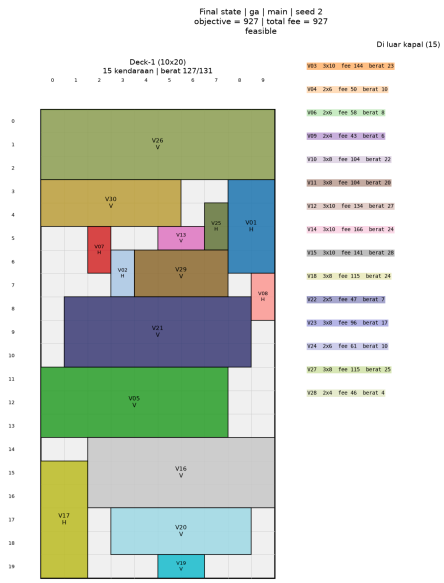
Tabel 2.14 *Visualisasi GA variasi generasi*

Konfigurasi	Visualisasi	Keterangan
Generasi 300, *seed* 2	 Gambar 2.35 *State Awal*	*State* awal -2.068 (individu terbaik populasi awal). Nilai akhir 863 (14 kendaraan, berat 122/131) baru ditemukan pada generasi ke-289, sehingga pencarian masih membaik saat dihentikan. Hanya 21 dari 100 individu akhir yang *feasible*.

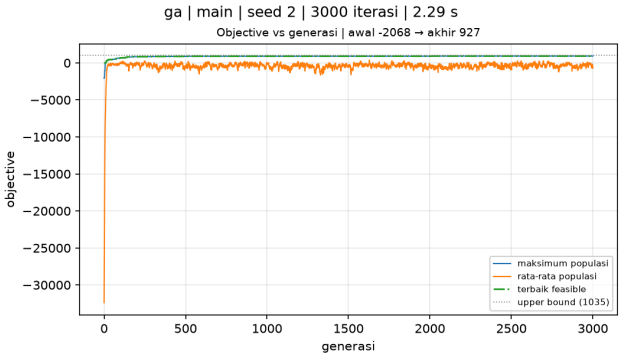
Konfigurasi	Visualisasi	Keterangan
	 Gambar 2.36 *State Akhir*	
	 Gambar 2.37 *Plot Maksimum dan Rata-rata Populasi*	

Konfigurasi	Visualisasi	Keterangan
Generasi 1.000, *seed* 2 (kontrol)	 Gambar 2.38 *State Awal*	Nilai akhir 908 (15 kendaraan, berat 127/131) ditemukan pada generasi ke-866. Rata-rata populasi akhir 52,1. Sebanyak 49 dari 100 individu *feasible*.
	 Gambar 2.39 *State Akhir*	

Konfigurasi	Visualisasi	Keterangan
	 Gambar 2.40 *Plot Maksimum dan Rata-rata Populasi*	
Generasi 3.000, *seed* 2	 Gambar 2.41 *State Awal*	Nilai akhir 927 (tertinggi pada kelompok ini) ditemukan pada generasi ke-2.012. Sekitar 1.000 generasi terakhir tanpa perbaikan. Rata-rata populasi akhir -716,3. Hanya 11 individu *feasible*.

Konfigurasi	Visualisasi	Keterangan
		

Gambar 2.42 *State Akhir*



Gambar 2.43 *Plot Maksimum dan Rata-rata Populasi*

Tabel 2.15 *Hasil seluruh run GA variasi generasi (populasi 100)*

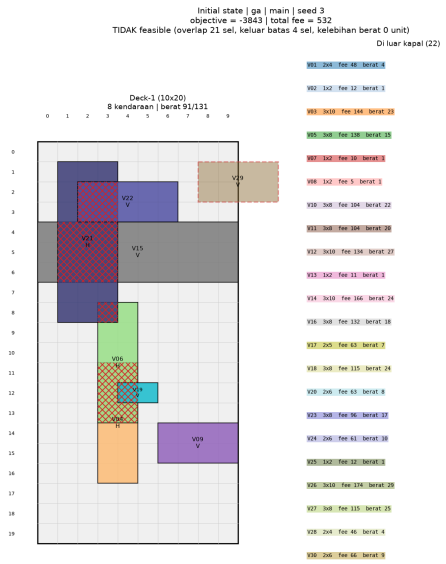
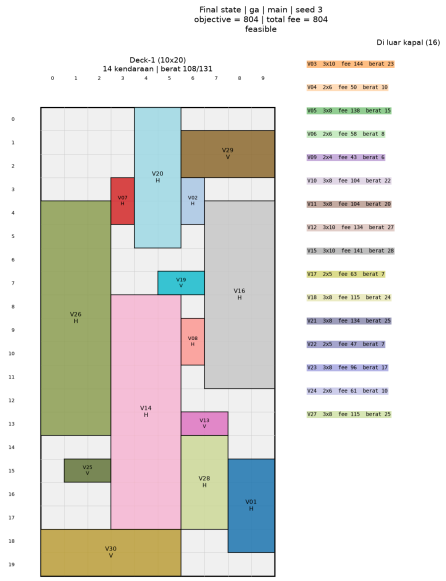
Generasi	*Seed*	*Objective* awal	*Objective* akhir	*Gap* UB	Generasi terbaik	Rata-rata populasi akhir	Individu *feasible* akhir	Durasi (s)
300	1	-7.019	721	30,3%	242	-9,9	42	0,284

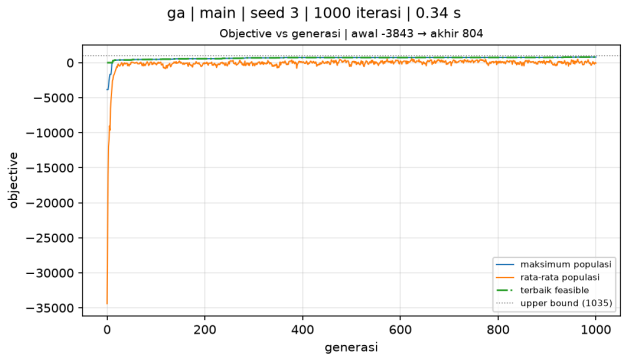
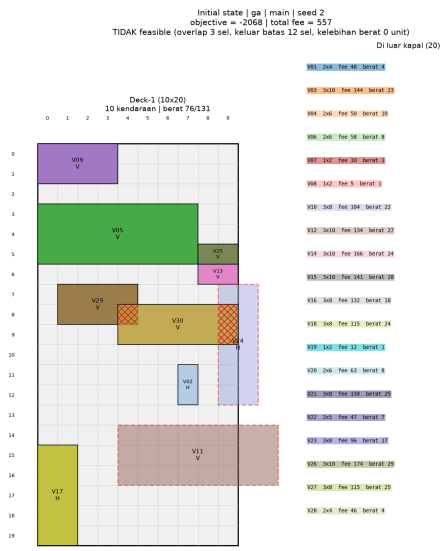
Generasi	*Seed*	*Objective* awal	*Objective* akhir	*Gap* UB	Generasi terbaik	Rata-rata populasi akhir	Individu *feasible* akhir	Durasi (s)
300	2	-2.068	863	16,6%	289	-410,9	21	0,250
300	3	-3.843	858	17,1%	227	-1.000,6	27	0,271
1.000	1	-7.019	780	24,6%	952	333,5	41	0,802
1.000	2	-2.068	908	12,3%	866	52,1	49	0,758
1.000	3	-3.843	865	16,4%	909	-277,1	59	0,856
3.000	1	-7.019	803	22,4%	1.528	-81,6	36	2,266
3.000	2	-2.068	927	10,4%	2.012	-716,3	11	2,292
3.000	3	-3.843	876	15,4%	2.520	-384,5	39	2,685

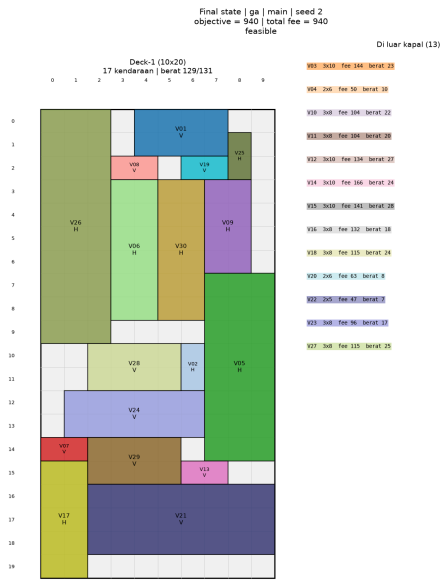
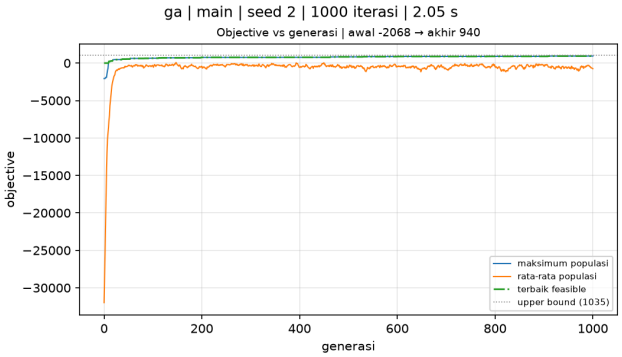
2.5.6.2 Variasi Jumlah Populasi (Generasi 1.000)

Gambar populasi 100 sama dengan Gambar 2.38-2.40.

Tabel 2.16 *Visualisasi GA variasi populasi*

Konfigurasi	Visualisasi	Keterangan
Populasi 50, *seed* 3		Nilai akhir 804 (14 kendaraan) dengan sisa kapasitas besar (berat 108/131, luas 72%). Ditemukan pada generasi ke-963. Sebanyak 21 dari 50 individu *feasible*.
	Gambar 2.44 *State Awal*	
		
	Gambar 2.45 *State Akhir*	

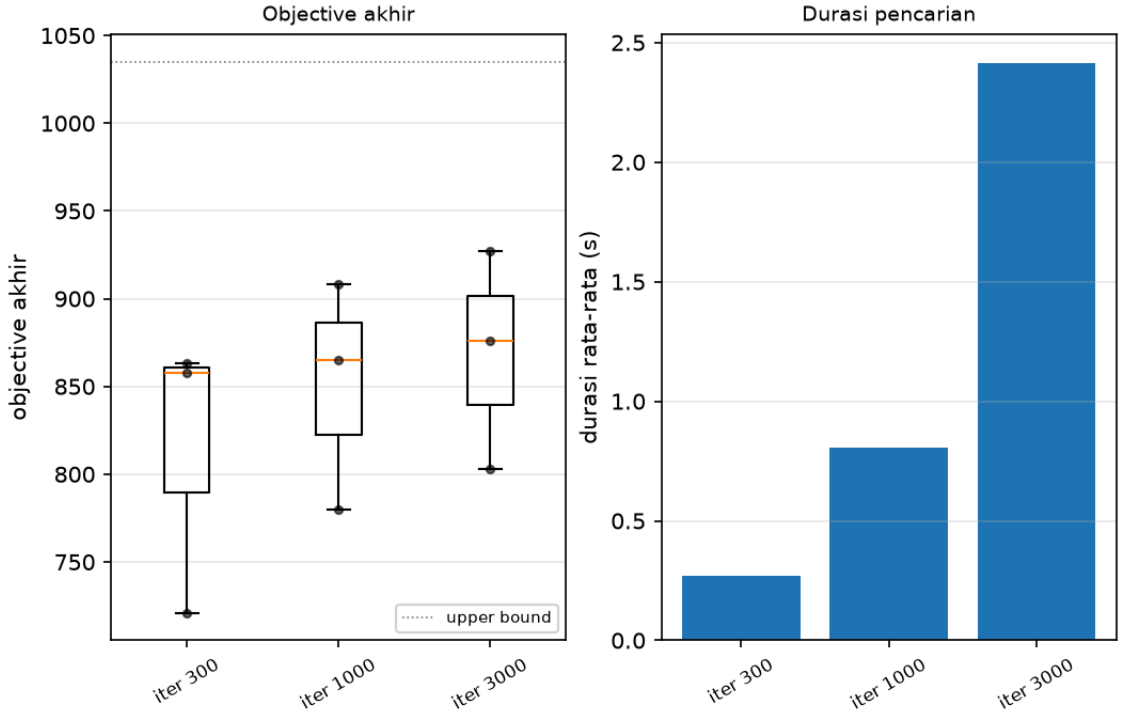
Konfigurasi	Visualisasi	Keterangan
	 Gambar 2.46 *Plot Maksimum dan Rata-rata Populasi*	
Populasi 200, *seed* 2	 Gambar 2.47 *State Awal*	Nilai akhir 940, tertinggi dari seluruh *run* GA (17 kendaraan, berat 129/131, luas 84%). Ditemukan pada generasi ke-909. Hanya 20 dari 200 individu akhir yang *feasible*.

Konfigurasi	Visualisasi	Keterangan
	 Gambar 2.48 *State Akhir*	
	 Gambar 2.49 *Plot Maksimum dan Rata-rata Populasi*	

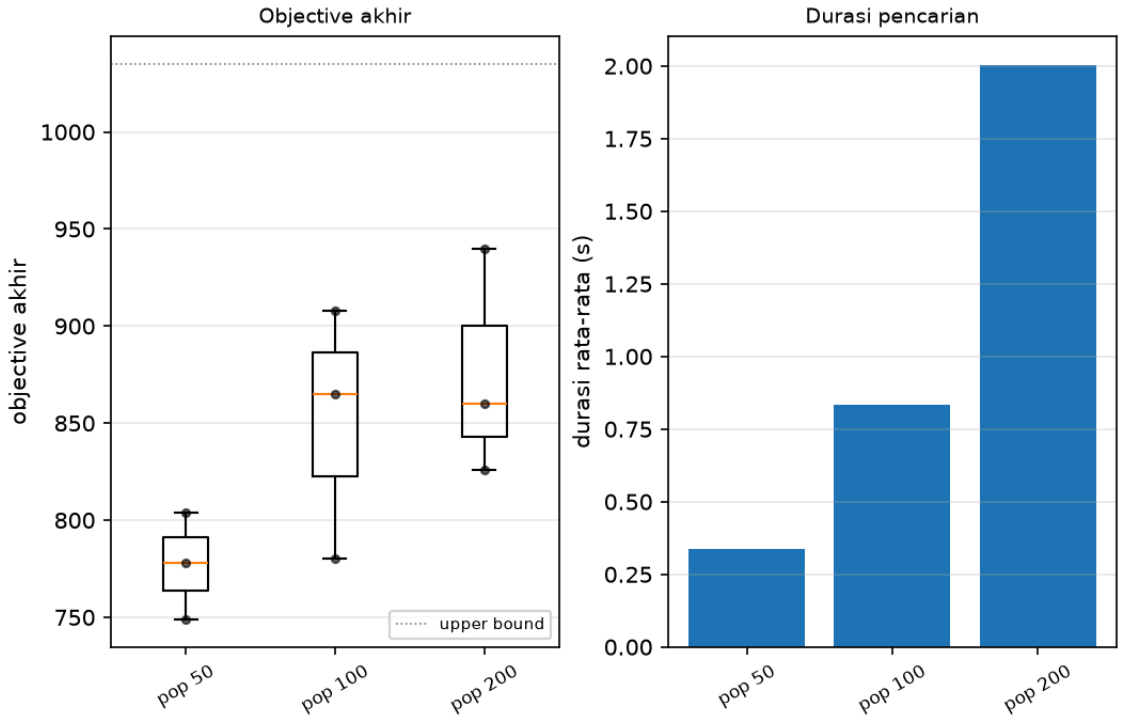
Tabel 2.17 *Hasil seluruh run GA variasi populasi (generasi 1.000)*

Populasi	*Seed*	*Objective* awal	*Objective* akhir	*Gap* UB	Generasi terbaik	Rata-rata populasi akhir	Individu *feasible* akhir	Durasi (s)
50	1	-7.748	749	27,6%	629	52,7	25	0,329

Populasi	*Seed*	*Objective* awal	*Objective* akhir	*Gap* UB	Generasi terbaik	Rata-rata populasi akhir	Individu *feasible* akhir	Durasi (s)
50	2	-12.585	778	24,8%	470	-28,5	22	0,341
50	3	-3.843	804	22,3%	963	-64,8	21	0,343
100	1	-7.019	780	24,6%	952	333,5	41	0,828
100	2	-2.068	908	12,3%	866	52,1	49	0,764
100	3	-3.843	865	16,4%	909	-277,1	59	0,913
200	1	-6.254	860	16,9%	872	-737,7	36	1,981
200	2	-2.068	940	9,2%	909	-747,8	20	2,054
200	3	-2.636	826	20,2%	886	-396,6	39	1,971



Gambar 2.50 *Sebaran objective akhir dan durasi GA variasi generasi*



Gambar 2.51 *Sebaran objective akhir dan durasi GA variasi populasi*

Tabel 2.18 *Ringkasan per konfigurasi GA*

Populasi	Generasi	*Objective* akhir (rata-rata ± simpangan baku)	Terbaik	*Gap* UB	Evaluasi	Durasi (s)
100	300	814,0 ± 80,6	863	21,4%	29.500	0,268
100	1.000	851,0 ± 65,1	908	17,8%	98.100	0,805
100	3.000	868,7 ± 62,3	927	16,1%	294.100	2,414
50	1.000	777,0 ± 27,5	804	24,9%	48.050	0,338
200	1.000	875,3 ± 58,5	940	15,4%	198.200	2,002

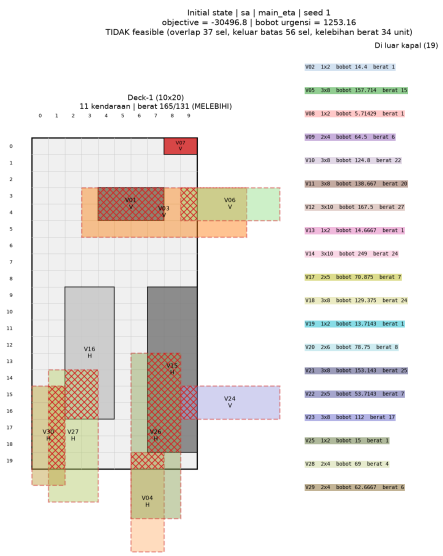
2.5.6.3 Analisis Genetic Algorithm

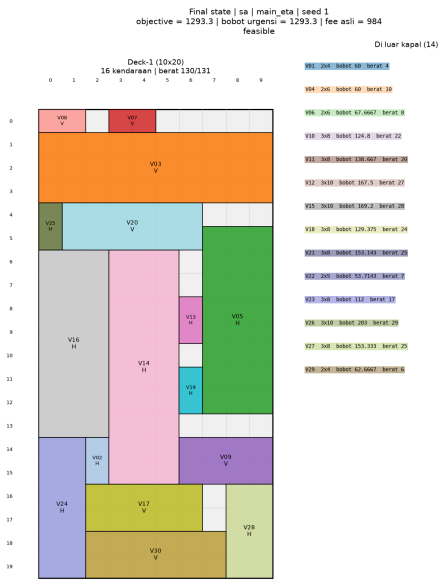
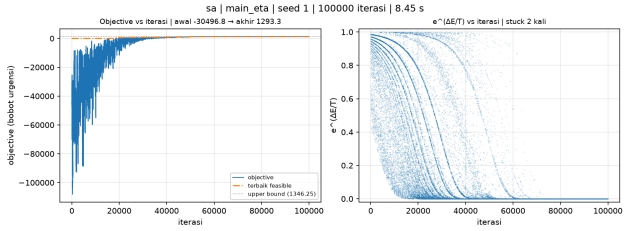
- Pengaruh banyak generasi. Menambah generasi selalu menaikkan hasil setiap *seed* karena *run* yang lebih panjang mengulang lintasan *run* yang lebih pendek, dan *elitism* menjaga nilai maksimum tidak turun. Namun, tambahannya makin kecil, yaitu +37,0 dari 300 ke 1.000 generasi dan hanya +17,7 dari 1.000 ke 3.000 generasi, padahal durasi naik sekitar 3 kali pada setiap langkah. Pada 300 generasi, nilai terbaik ditemukan pada generasi ke-227-289 (masih terpotong).
- Pengaruh jumlah populasi. Rata-rata naik dari 777,0 (50) ke 851,0 (100) dan 875,3 (200), karena populasi besar memuat lebih banyak variasi susunan untuk digabungkan. Kenaikan ini tidak terjadi pada semua *seed* (*seed* 3: 865 → 826), karena ukuran populasi mengubah seluruh urutan bilangan acak. Dengan biaya serupa, populasi 200 x 1.000 generasi (198.200 evaluasi, 2,0 detik) lebih baik daripada populasi 100 x 3.000 generasi (294.100 evaluasi, 2,4 detik).
- Rata-rata populasi tetap rendah. Rata-rata populasi naik cepat dari sekitar -30.000, tetapi kemudian berfluktuasi jauh di bawah nilai maksimum. Pada generasi terakhir hanya 10-59% individu yang *feasible*. Dugaan penyebabnya: (1) *crossover* satu titik menukar koordinat antarinduk sehingga anak dari dua induk *feasible* dapat bertumpuk, (2) mutasi satu langkah acak lebih sering merusak susunan padat daripada memperbaikinya, dan (3) *windowing* terhadap individu terburuk yang berpenalti besar membuat selisih *fitness* antarindividu baik relatif kecil sehingga tekanan seleksi melemah. Dugaan ini belum diverifikasi dengan uji ablasi.
- Dibanding algoritma lain. Konfigurasi GA terbaik (875,3) berada di bawah Steepest Ascent (954,0) dan SA (966,3), walaupun jumlah evaluasinya setara dengan Steepest Ascent. Pada persoalan penempatan ini, perbaikan lokal yang sistematis lebih efektif daripada penggabungan dua susunan yang posisinya tidak saling menyesuaikan.

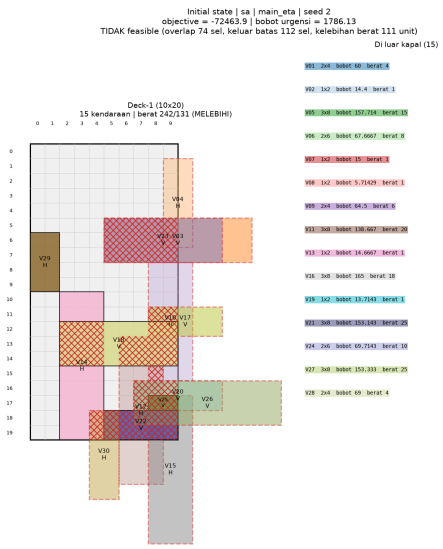
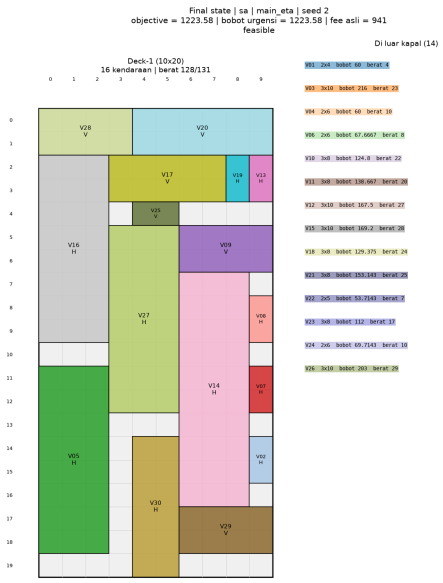
2.5.7 Bonus 1: *Objective* Urgensi ETA

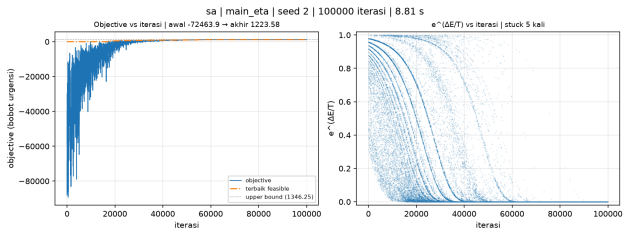
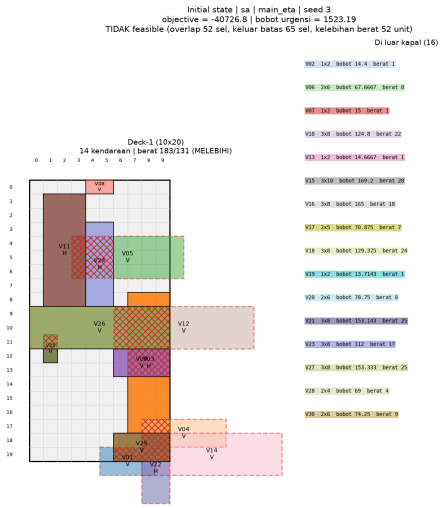
SA dijalankan dengan *objective* ETA, parameter dan *seed* yang sama, serta *initial state* yang sama dengan Subbab 2.5.5. Nilai *objective* adalah total bobot urgensi, bukan pendapatan. UB ETA = 1.346,25.

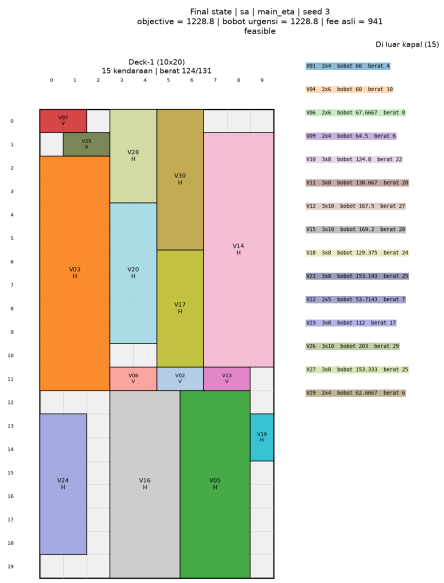
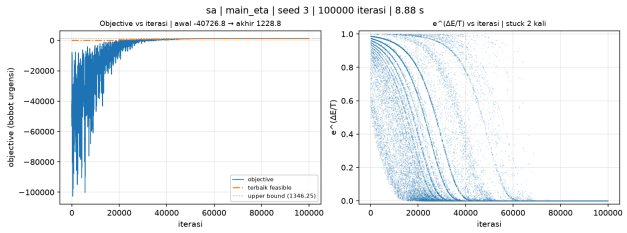
Tabel 2.19 *Visualisasi SA dengan objective ETA*

Trial	**Visualisasi**	**Keterangan**
1	 Gambar 2.52 *State Awal*	Nilai akhir 1.293,3 (*fee* asli 984): 16 kendaraan, berat 130/131, luas 91%. Dua episode *stuck*.

Trial	Visualisasi	Keterangan
	 Gambar 2.53 *State Akhir*	
	 Gambar 2.54 *Plot Objective dan* $e^{\Delta E/T}$	

Trial	Visualisasi	Keterangan
2	 Gambar 2.55 *State Awal*	Nilai akhir 1.223,6 (*fee* asli 941): 16 kendaraan, berat 128/131. Lima episode *stuck*, terbanyak di antara semua *run* SA.
	 Gambar 2.56 *State Akhir*	

Trial	Visualisasi	Keterangan
	 Gambar 2.57 *Plot Objective dan* $e^{\Delta E/T}$	
3	 Gambar 2.58 *State Awal*	Nilai akhir 1.228,8 (*fee* asli 941): 15 kendaraan, berat 124/131. Dua episode *stuck*.

Trial	Visualisasi	Keterangan
	 Gambar 2.59 *State Akhir*	
	 Gambar 2.60 *Plot Objective dan* $e^{\Delta E/T}$	

Tabel 2.20 *Perbandingan objective fee dan ETA pada seed yang sama (SA)*

Seed	***Objective* pencarian**	***Fee* asli**	**Bobot urgensi**	**Dimuat**	**Rata-rata ETA dimuat**	**Durasi (s)**
1	*fee*	964	1.243,5	11	3,82	8,435
1	ETA	984	1.293,3	16	3,63	8,455
2	*fee*	973	1.206,8	15	4,20	8,488
2	ETA	941	1.223,6	16	3,44	8,808

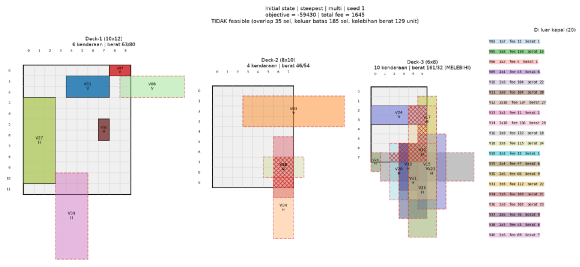
Seed	*Objective* pencarian	*Fee* asli	Bobot urgensi	Dimuat	Rata-rata ETA dimuat	Durasi (s)
3	*fee*	962	1.148,1	17	4,29	10,121
3	ETA	941	1.228,8	15	3,80	8,883

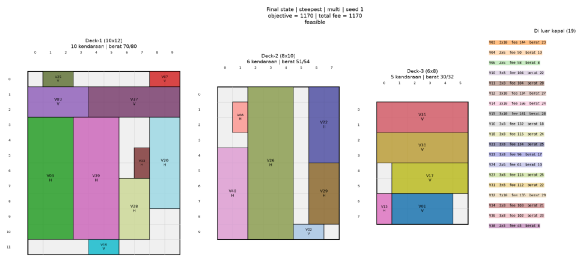
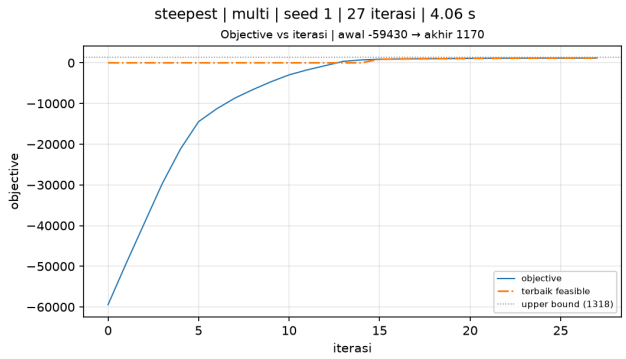
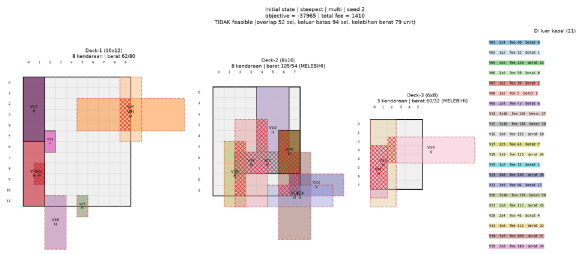
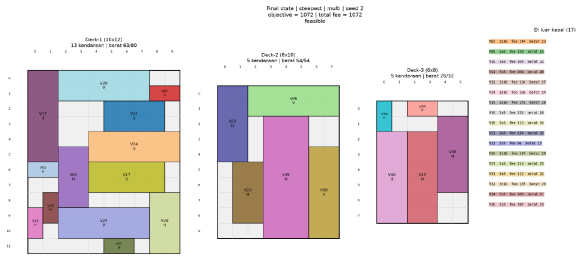
Analisis. *Objective* ETA mencapai tujuannya pada ketiga *seed*: dibanding *run objective fee*, total bobot urgensi naik 16,8-80,7 dan rata-rata ETA kendaraan yang dimuat turun (misalnya 4,20 → 3,44). Pendapatan asli turun rata-rata 11,0 (966,3 → 955,3) karena kendaraan ber-*fee* tinggi yang tidak mendesak ditukar dengan kendaraan yang lebih mendesak. T_0 pada *objective* ETA sekitar 1,43 kali T_0 *objective fee*, sama dengan rasio penalti 250/175, yang menunjukkan bahwa ΔE awal didominasi penalti dan heuristik T_0 otomatis menyesuaikan skala tanpa pengaturan ulang. *Gap* terhadap UB ETA 3,9-9,1%, dengan simpangan baku 38,8.

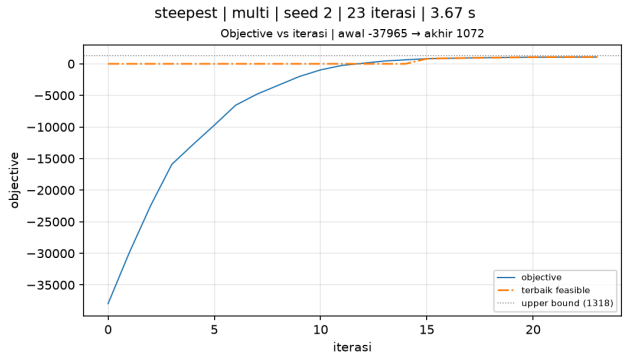
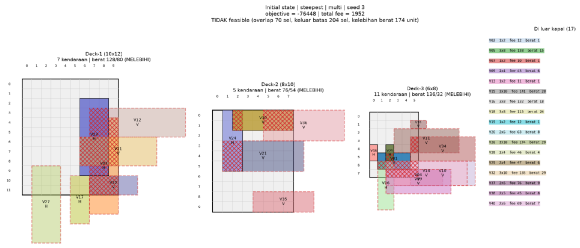
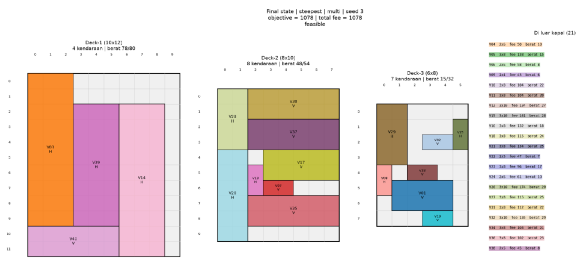
2.5.8 Bonus 3: Multikapal

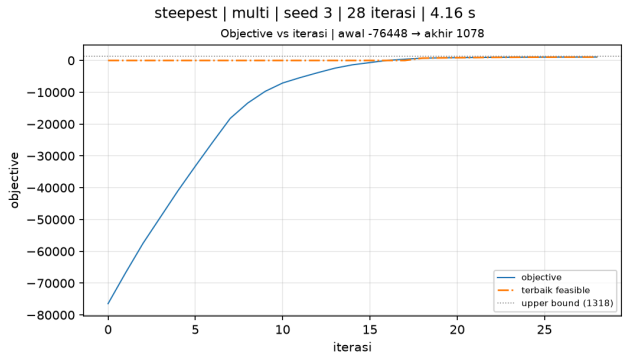
Steepest Ascent dijalankan pada `multi.json` (tiga kapal, 40 kendaraan, UB = 1.318).

Tabel 2.21 *Visualisasi multikapal*

Trial	Visualisasi	Keterangan
1	 Gambar 2.61 *State Awal*	*State* awal -59.430. Berhenti di *local optimum* pada iterasi ke-27 dengan nilai 1.170 (terbaik): 21 kendaraan (10/6/5 per kapal), berat 70/80, 51/54, 30/32.

Trial	Visualisasi	Keterangan
	 Gambar 2.62 *State Akhir*	
	 Gambar 2.63 *Plot Objective*	
2	 Gambar 2.64 *State Awal*	Nilai 1.072 pada iterasi ke-23: 23 kendaraan (13/5/5), berat 63/80, 54/54, 26/32.
	 Gambar 2.65 *State Akhir*	

Trial	Visualisasi	Keterangan
	 Gambar 2.66 *Plot Objective*	
3	 Gambar 2.67 *State Awal*	Nilai 1.078 pada iterasi ke-28: 19 kendaraan (4/8/7), berat 78/80, 48/54, 15/32.
	 Gambar 2.68 *State Akhir*	

Trial	Visualisasi	Keterangan
		

Gambar 2.69 *Plot Objective*

Tabel 2.22 *Hasil multikapal*

Trial	*Objective* awal	*Objective* akhir	*Gap* UB	Iterasi	Evaluasi	Durasi (s)
1	-59.430	1.170	11,2%	27	569.887	4,060
2	-37.965	1.072	18,7%	23	489.470	3,666
3	-76.448	1.078	18,2%	28	590.444	4,156
Rata-rata ± simpangan baku		1.106,7 ± 54,9	16,0%			3,961 ± 0,259

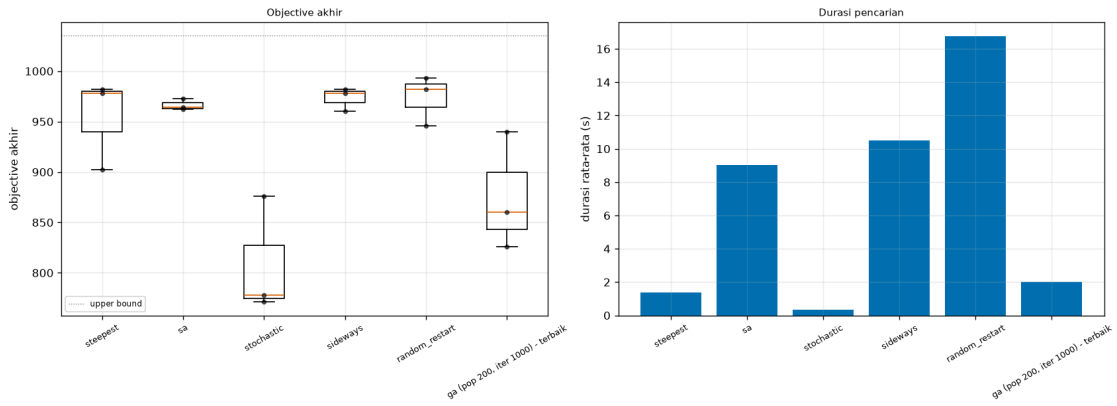
Analisis. Semua *run feasible* pada ketiga kapal, dengan *occupancy* dan kapasitas diperiksa per kapal. *Gap* lebih besar daripada instans satu kapal karena dua hal. Pertama, UB menggabungkan kapasitas dan luas seluruh armada, padahal muatan nyata harus terbagi per kapal. Misalnya, bus 3 x 10 tidak muat di Deck-3 (6 x 8). Kedua, *neighborhood* lebih besar (tiga geladak) sehingga lebih banyak *local optimum*. Pemanfaatan kapal berbeda-beda antar-*trial* (contoh: Deck-3 hanya 15/32 pada *trial* 3), yang diduga karena Steepest Ascent sulit memindahkan muatan antarkapal setelah setiap kapal terisi.

2.5.9 Perbandingan Antaralgoritma

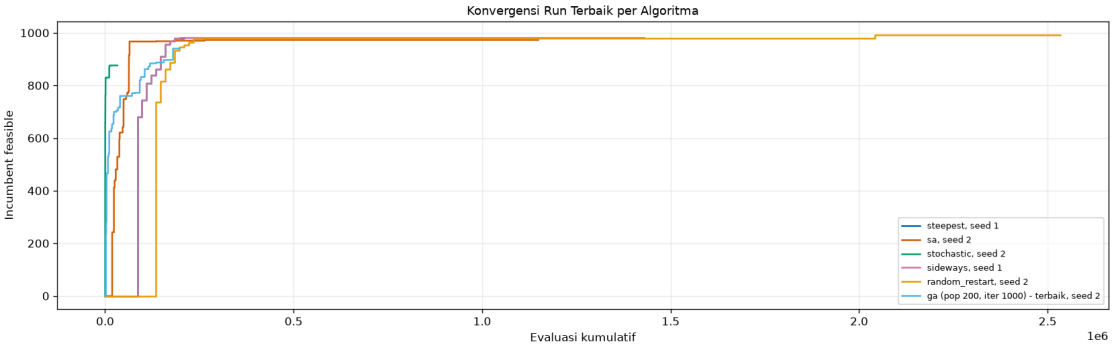
Tabel 2.23 *Ringkasan pada* `main.json` *(3 seed)*

Algoritma	Terbaik	Rata-rata ± simpangan baku	*Gap* UB rata-rata	Durasi rata-rata (s)	Evaluasi rata-rata
Steepest Ascent HC	982	954,0 ± 45,1	7,8%	1,376	229.241
Stochastic HC	876	808,3 ± 58,7	21,9%	0,349	31.168
Sideways Move HC	982	973,3 ± 11,7	6,0%	10,523	1.454.800
Random Restart HC	993	973,7 ± 24,6	5,9%	16,745	2.534.385
Simulated Annealing	973	966,3 ± 5,9	6,6%	9,015*	1.136.824*
GA (populasi 200, generasi 1.000)	940	875,3 ± 58,5	15,4%	2,002	198.200

*Termasuk pemeriksaan *stuck*. Pencarian SA saja: 100.101 evaluasi dan sekitar 1,2 detik.



Gambar 2.70 *Sebaran objective akhir dan rata-rata durasi setiap algoritma*



Gambar 2.71 *Konvergensi state feasible terbaik terhadap evaluasi kumulatif (run terbaik tiap algoritma)*

1. Kedekatan dengan *global optimum*. Optimum eksak `main.json` tidak dapat dihitung, sehingga dipakai UB = 1.035. Karena optimum ≤ UB, *gap* adalah batas atas jarak ke optimum. Nilai terbaik yang diketahui adalah 993 (*gap* ≤ 4,1%), sehingga *global optimum* berada di antara 993 dan 1.035. Random Restart, Sideways, dan SA rata-rata berada dalam 5,9–6,6% dari UB, sedangkan Stochastic HC dan GA berada pada 15–22%. Pada `small.json`, UB (298) hanya 3,8% di atas optimum eksak (287), sehingga relaksasi ini cukup ketat. Hasil hampir semua *run* terbaik memakai berat 130–131/131 dengan luas 80–90%: kapasitas berat menjadi kendala yang paling mengikat, dan kendaraan ber-*fee* tinggi tetapi berat (misalnya V03 *fee* 144/berat 23 dan V15 141/28) sering tidak dimuat.
2. Perbandingan hasil. Urutan rata-rata: Random Restart ≈ Sideways > SA > Steepest Ascent > GA > Stochastic. Algoritma yang memindai seluruh *neighborhood* atau mampu keluar dari *local optimum* unggul. Stochastic HC dan GA lemah karena proposal acak jarang menemukan perbaikan pada susunan padat dan *crossover* merusak susunan.
3. Durasi. Stochastic HC paling cepat (0,35 detik), disusul Steepest Ascent (1,4 detik) dan GA (0,3–2,4 detik bergantung konfigurasi). Sideways (10,5 detik) dan Random Restart (16,7 detik) mahal karena mengulang pemindaian *neighborhood* penuh. Durasi SA tercatat 9,0 detik, tetapi pencarian intinya hanya sekitar 1,2 detik. Sisanya merupakan biaya pemeriksaan *stuck*. Dengan durasi setara Steepest Ascent, SA memberi hasil yang lebih konsisten.

4. Konsistensi. Simpangan baku terkecil: SA (5,9), lalu Sideways (11,7), Random Restart (24,6), Steepest (45,1), dan Stochastic (58,7), sedangkan GA berkisar 27,5-80,6 bergantung konfigurasi. Hill-Climbing yang hanya menerima perbaikan (Steepest, Stochastic) sangat bergantung pada *initial state*, sedangkan SA mengurangi ketergantungan tersebut. Tiga *seed* hanya memberi indikasi, bukan kesimpulan statistik.
5. Pengaruh parameter GA. Generasi dan populasi sama-sama menaikkan hasil dengan *diminishing return*. Pada anggaran evaluasi yang serupa, menambah populasi lebih efektif daripada menambah generasi (Subbab 2.5.6.3).

Kesimpulan dan Saran

3.1. Kesimpulan

Berdasarkan implementasi dan eksperimen yang telah dilakukan, kami menarik beberapa kesimpulan berikut.

1. Enam algoritma *local search* (Steepest Ascent, Stochastic, Sideways Move, Random Restart HC, Simulated Annealing, dan Genetic Algorithm) berhasil diimplementasikan dengan representasi, *neighbor*, dan *objective* yang sama. Seluruh 39 *run* menghasilkan *state* akhir yang memenuhi semua *constraint*, yaitu penalti $P = fee\,maks + 1$ mengubah *initial state* acak yang tidak *feasible* menjadi susunan valid.
2. Pada instans utama, rata-rata terbaik dicapai Random Restart HC (973,7) dan Sideways Move (973,3), disusul SA (966,3) dan Steepest Ascent (954,0). Nilai terbaik 993 hanya 4,1% di bawah *upper bound*, sehingga jaraknya ke *global optimum* tidak lebih dari 4,1%.
3. Steepest Ascent cepat (sekitar 1,4 detik), tetapi mudah terjebak di *local optimum* yang ditentukan *initial state* (902–982). *Sideways move* membantu keluar dari *plateau* hanya pada satu dari tiga *trial*, sedangkan *restart* memperbaiki hasil dengan biaya sebanding banyaknya percobaan. Stochastic HC paling cepat, tetapi hasilnya paling rendah.
4. SA paling konsisten (simpangan baku 5,9). T_0 otomatis menyesuaikan skala penalti, dan *stuck* hanya terjadi saat suhu rendah (1–4 episode setelah iterasi ke-59.000).
5. GA membaik dengan bertambahnya generasi dan populasi, tetapi tetap di bawah HC dan SA karena *crossover* koordinat menghasilkan banyak anak yang tidak *feasible*.
6. Bonus: *objective* ETA berhasil memprioritaskan kendaraan mendesak dengan pengorbanan pendapatan rata-rata 11,0 dan Steepest Ascent multikapal menghasilkan susunan *feasible* pada tiga kapal (rata-rata 1.106,7, *gap* 16,0%).

3.2. Saran

Beberapa hal yang dapat dikembangkan pada pekerjaan berikutnya adalah sebagai berikut.

1. T_0 SA sebaiknya diambil dari *state* yang sudah *feasible* atau dibatasi skalanya. T_0 dari *initial state* berpenalti membuat sepertiga awal iterasi hampir berupa *random walk*, dan sepertiga akhir tidak lagi mengubah hasil.
2. Pemeriksaan *stuck* dapat dibuat lebih jarang atau berbasis *sampling* karena memakan sekitar 85% durasi SA.
3. GA dapat memakai *crossover* yang mempertahankan kelayakan (misalnya memilih subset kendaraan, lalu menempatkannya dengan *repair* lokal) atau menggunakan *tournament selection* untuk memperkuat tekanan seleksi.
4. Perbandingan algoritma sebaiknya juga dilakukan pada anggaran evaluasi yang sama dan dengan lebih banyak *seed* agar konsistensi dapat diuji secara statistik.
5. *Upper bound* yang lebih ketat (misalnya relaksasi per kapal untuk multikapal) akan memberi estimasi jarak ke optimum yang lebih akurat.

Referensi

[1] Slide Perkuliahan IF3070 Dasar Inteligensi Artifisial

[2] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: Beyond Classical Search: Classical vs Local Search [Accessed: Oct. 22, 2026].

[3] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Oct. 22, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Oct. 22, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: Beyond Classical Search: Simulated Annealing [Accessed: Oct. 22, 2026].

[6] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: Beyond Classical Search: Genetic Algorithm [Accessed: Oct. 22, 2026].

[7] Python Software Foundation, "bisect — Array bisection algorithm," Python 3 Documentation. Available: https://docs.python.org/3/library/bisect.html. [Accessed: Oct. 22, 2026].

[8] The Matplotlib Development Team, "Matplotlib: Visualization with Python." Available: https://matplotlib.org. [Accessed: Oct. 22, 2026].

Lampiran

A. Pembagian Tugas

NIM	Nama	Pekerjaan
18224057	Levina Nathania Bunardi	Visualisasi, CLI, runner eksperimen, notebook, koordinasi laporan
18224059	Tiara Kusuma Wardhani	Simulated Annealing, Genetic Algorithm, *objective* ETA (Bonus 1)
18224061	Riantama Putra	Input/generator, representasi *state*, *objective* dan *delta evaluation*, *neighbor*, *upper bound*, Hill-Climbing beserta varian (Bonus 2), multikapal (Bonus 3)

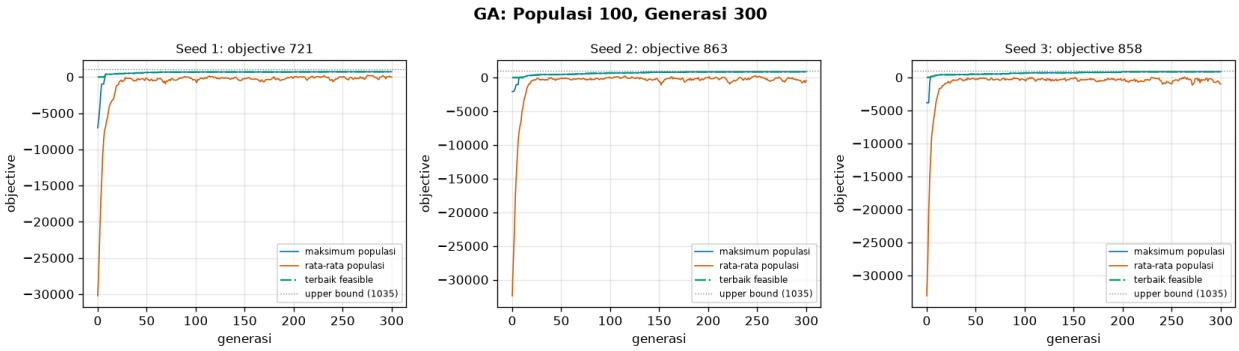
B. Github Repository

Link : https://github.com/veinsan/IF3070_G09_Sepakat/

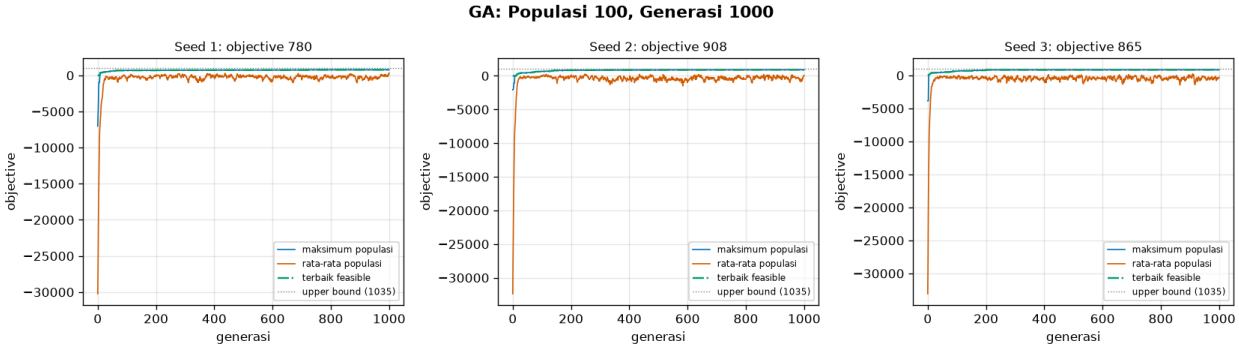
C. Gambar Run GA Lainnya

Setiap run memiliki tiga gambar `_initial.png`, `_final.png`, dan `_plot.png` pada folder berikut.

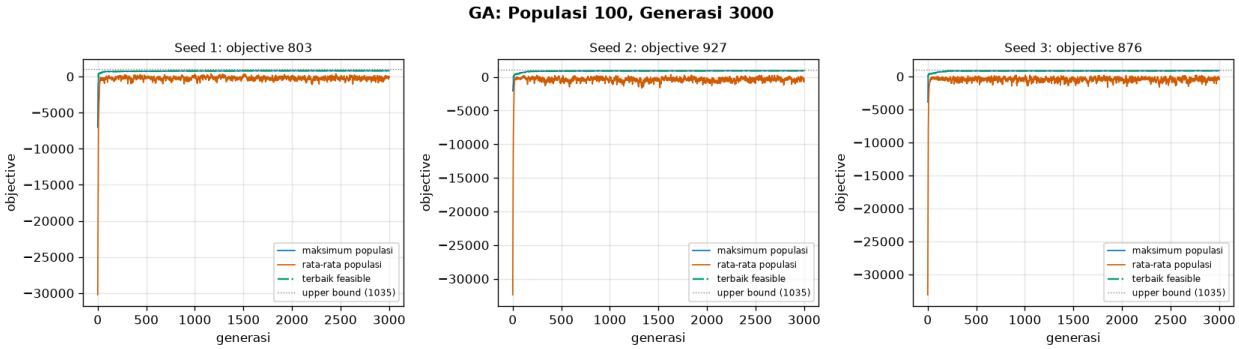
Konfigurasi	Seed	Prefix berkas
Populasi 100, generasi 300	1, 3	`output/experiments/ga_iter/ga/main_pop100_iter300_seed{1,3}`
Populasi 100, generasi 1.000	1, 3	`output/experiments/ga_iter/ga/main_pop100_iter1000_seed{1,3}`
Populasi 100, generasi 3.000	1, 3	`output/experiments/ga_iter/ga/main_pop100_iter3000_seed{1,3}`
Populasi 50, generasi 1.000	1, 2	`output/experiments/ga_pop/ga/main_pop50_iter1000_seed{1,2}`
Populasi 200, generasi 1.000	1, 3	`output/experiments/ga_pop/ga/main_pop200_iter1000_seed{1,3}`



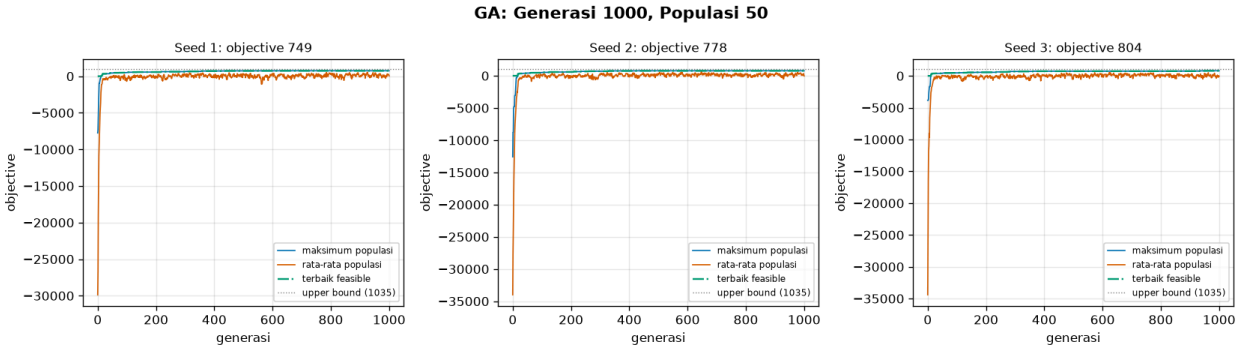
Gambar C.1 *GA populasi 100, generasi 300 (tiga seed)*



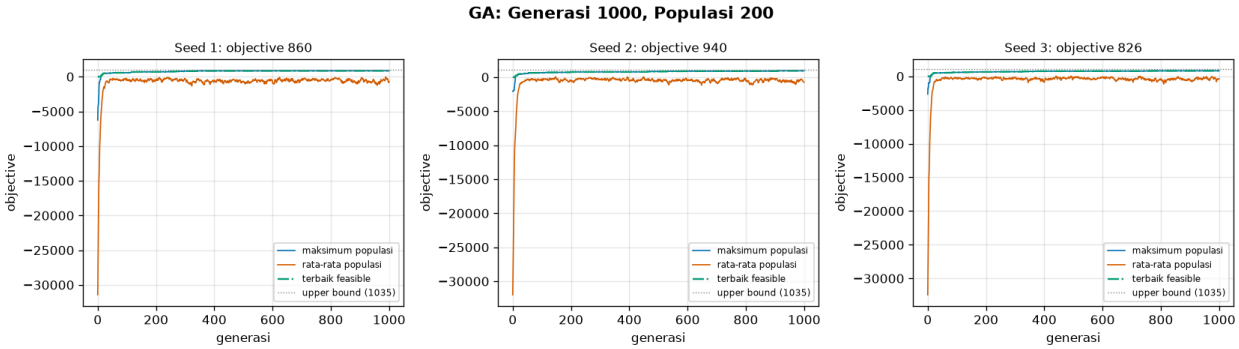
Gambar C.2 *GA populasi 100, generasi 1.000 (tiga seed)*



Gambar C.3 *GA populasi 100, generasi 3.000 (tiga seed)*

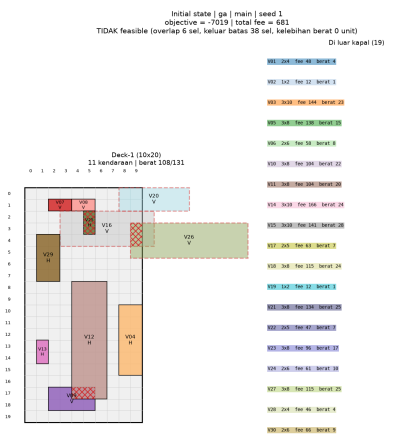
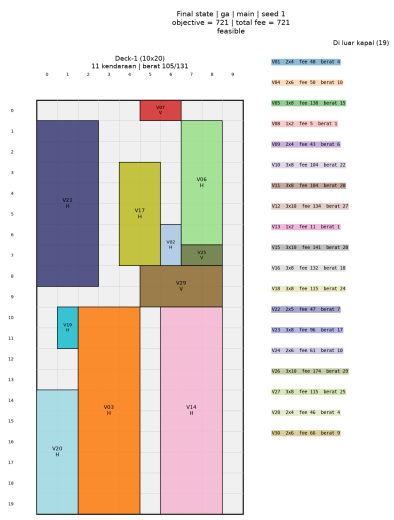


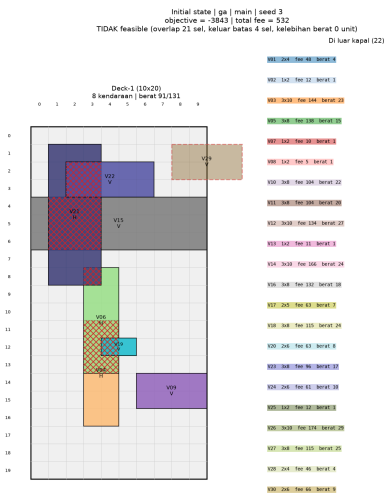
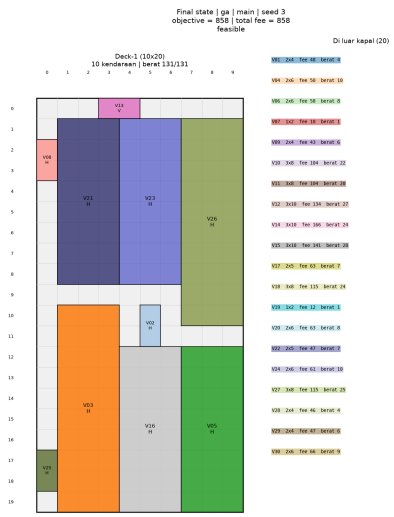
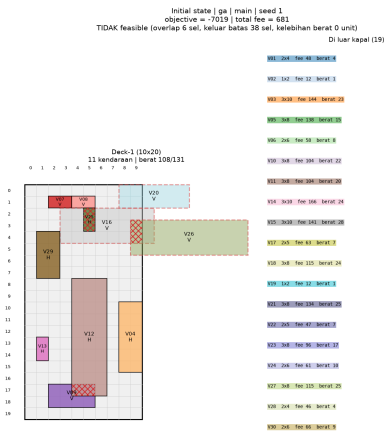
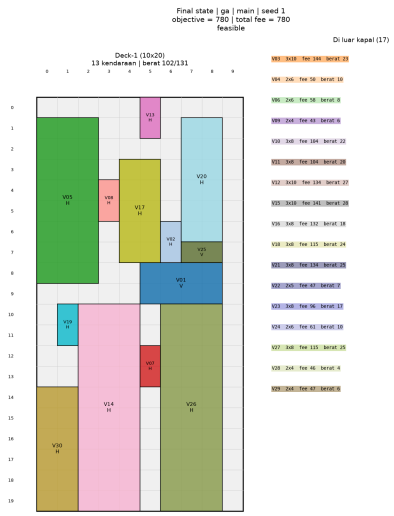
Gambar C.4 *GA populasi 50, generasi 1.000 (tiga seed)*

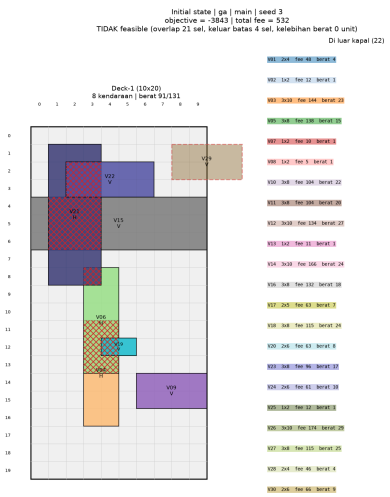
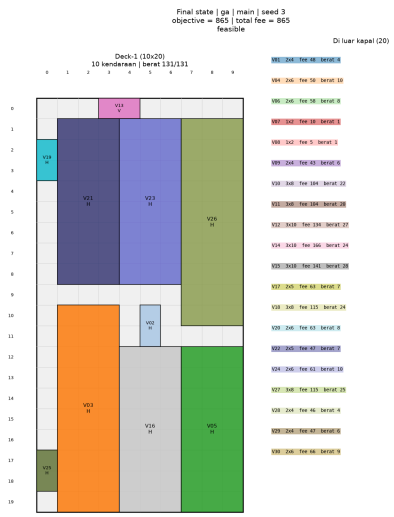
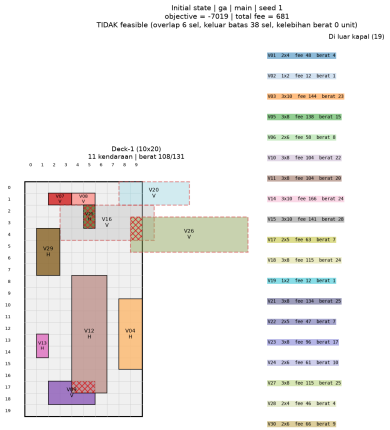
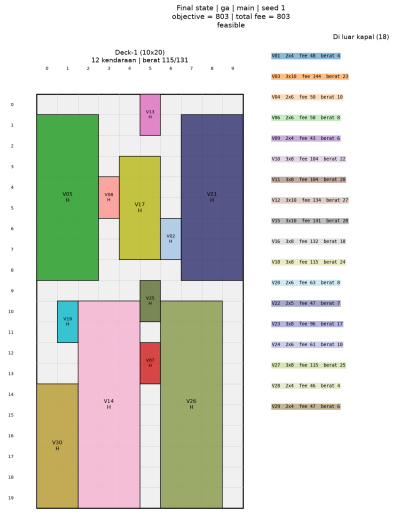


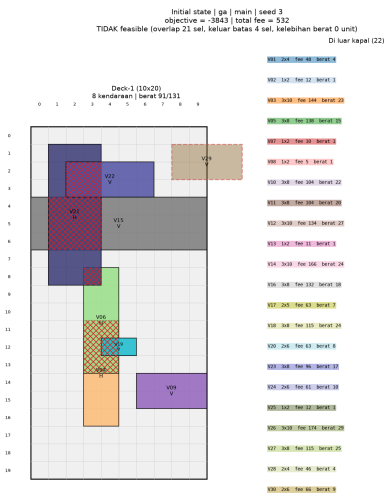
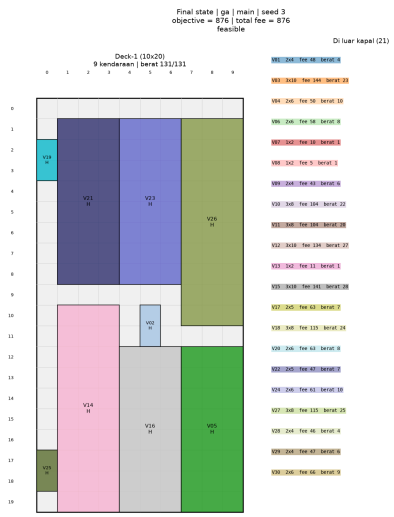
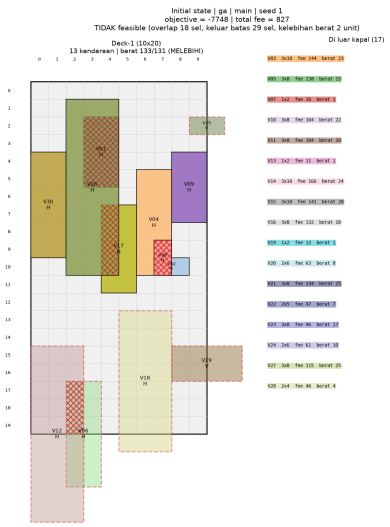
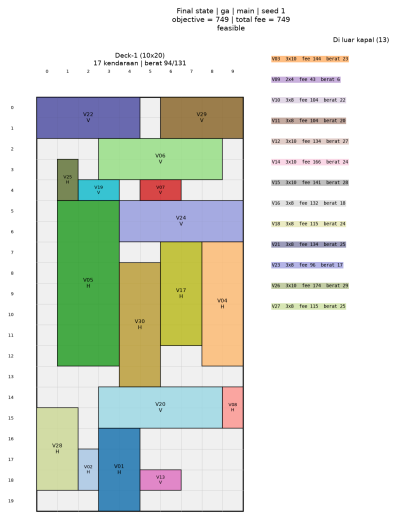
Gambar C.5 *GA populasi 200, generasi 1.000 (tiga seed)*

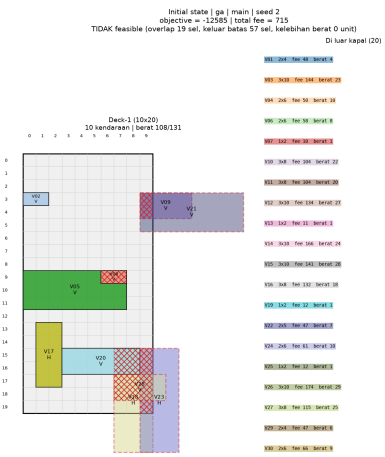
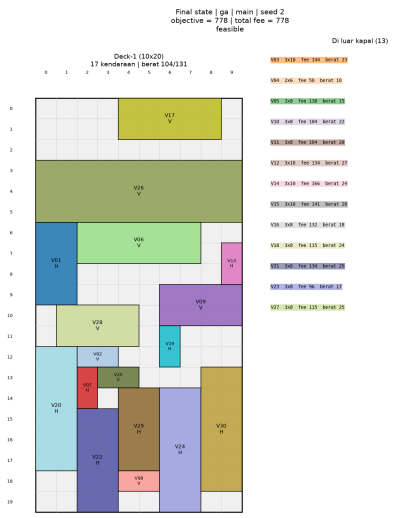
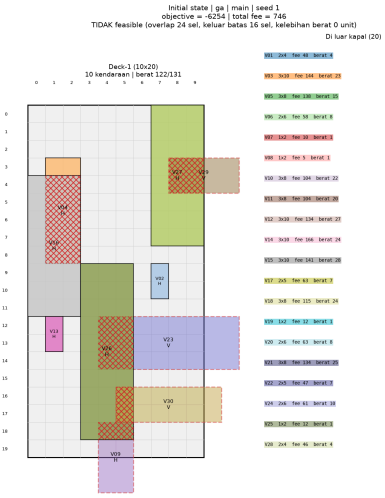
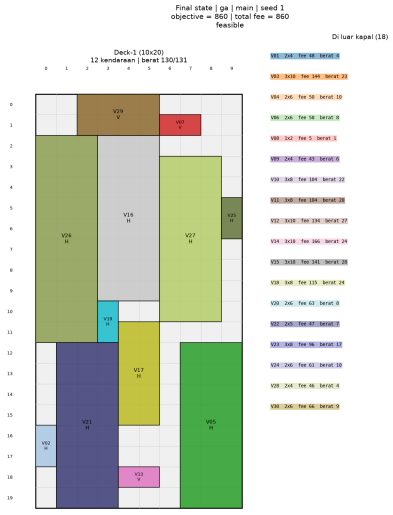
Tabel C.1 *State awal dan akhir run GA yang tidak ditampilkan pada Bab 2*

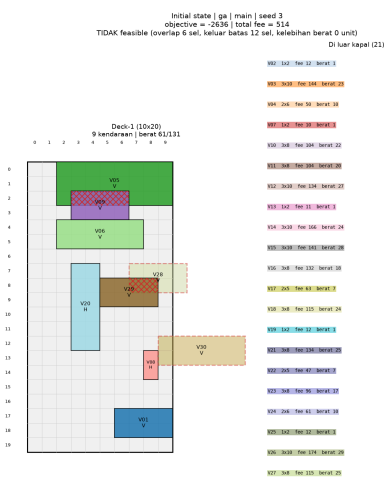
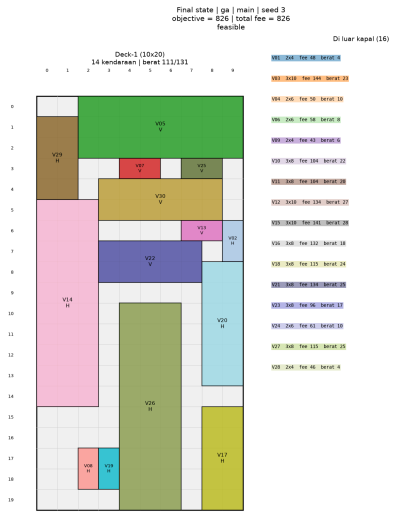
Konfigurasi	***Seed***	***State* awal**	***State* akhir**
Populasi 100, generasi 300	1	 Gambar C.6	 Gambar C.7

Konfigurasi	*Seed*	*State* awal	*State* akhir
Populasi 100, generasi 300	3	 Gambar C.8	 Gambar C.9
Populasi 100, generasi 1.000	1	 Gambar C.10	 Gambar C.11

Konfigurasi	*Seed*	*State* awal	*State* akhir
Populasi 100, generasi 1.000	3	 Gambar C.12	 Gambar C.13
Populasi 100, generasi 3.000	1	 Gambar C.14	 Gambar C.15

Konfigurasi	*Seed*	*State* awal	*State* akhir
Populasi 100, generasi 3.000	3	 Gambar C.16	 Gambar C.17
Populasi 50, generasi 1.000	1	 Gambar C.18	 Gambar C.19

Konfigurasi	*Seed*	*State* awal	*State* akhir
Populasi 50, generasi 1.000	2	 Gambar C.20	 Gambar C.21
Populasi 200, generasi 1.000	1	 Gambar C.22	 Gambar C.23

Konfigurasi	*Seed*	*State* awal	*State* akhir
Populasi 200, generasi 1.000	3	 Gambar C.24	 Gambar C.25

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:
Nama/NIM : Levina Nathania Bunardi/18224057 Tiara Kusuma Wardhani/18224059 Riantama Putra/18224061
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah : IF3070
Nama Mata Kuliah : Dasar Inteligensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	No AI (0%)
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	AI-assisted drafting - human editing and verification (30%)
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	AI-assisted idea generation and structuring (40%)
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	No AI
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Riantama Putra
18224061